National University of Science and Technology

Think in Other Terms

Faculty of Engineering

Department of Electronic Engineering

Name	Francois Mavunila
Student Number	N0172507W
Course Code	TEE5003
Project Title	Taxi safety and robbery prevention system
Supervisor	Mr. B. Dlodlo

Declaration

I, Francois Mavunila hereby declare that the work presented in this document is a result of my work, done under the supervision of Mr. B.Dlodlo and all the information obtained from other people's works is properly referenced. The work done in this project has not been submitted to any other university or Institution.

Signature----------

Date----------

ACKNOWLEDGEMENT

I would like to thank everyone who helped me in doing this project from start to end, special thanks to my project supervisor Mr. B Dlodlo for giving me a chance and guidance in doing this project. I would also want to give thanks to my friends and family who supported me with the finances and motivation to complete my project.

ABSTRACT

Taxis are of use in our day to day lives, they help carry people from one place to another especially in areas that are not in the route of minibuses and Buses, however there has been a concern with the rise of taxi related robberies in the country. Both passengers and drivers fall victim of such incidents and police find it difficult to trace the robbers and in some cases the victims are found dead and dumped after some days of efforts in trying to find them. Most of these robberies happen at night where the victim cannot identify the robbers and the vehicle plate number and hence investigations become difficult. The purpose of this project is to develop a system that will help in robbery investigation in taxi related robberies.

The whole objective of this project is to develop a device that will allow the driver and the passengers to authenticate their taxi ride using their fingerprints. The fingerprints will be kept in a secure database and used for investigation in case of any robbery.

The system was implemented successfully, the information was sent via MQTT communication protocol and a web application was added to the system to monitor the activities and manage users and the database.

Testing of the system was done using a number of selected individuals and the results showed that the system could be used in real life situation in Zimbabwe.

The project is easy to install and use and does actually help in investigating robberies and in reducing taxi related robberies.

Table of Contents

1 INTRODUCTION 9
1.1 Introduction 9
1.2 Background 9
1.3 Problem Statement 11
1.4 Aim 11
1.5 Objectives 11
1.6 Justification 12
2 LITERATURE REVIEW 13
2.1 Introduction 13
2.2 Public transport industry 13
2.2.1 Taxis 14
2.2.2 Transport system in Zimbabwe 16
2.3 Safety issues in transport 17
2.3.1 Causes of crimes 18
2.4 Safety issues in Zimbabwe 18
2.5 Solutions 19
2.5.1 World wide solutions 19
2.5.2 Solutions in Zimbabwe 23
2.6 System Technology 23
2.6.1 Characteristics Of fingerprints: 23
2.6.2 Fingerprint sensors 24
2.6.3 Internet of Things 25
2.6.4 MQTT 26
2.6.5 HTTP 27
2.6.6 Raspberry Pi 4 board 27
2.6.7 Databases 31
2.6.8 APIs 32
2.6.9 Single Page Applications 32
2.7 Conclusion 33
3 Methodology 34
3.1 Introduction 34

3.2 Materials used 34
3.3 System high level design 35
3.4 Design of subsystems 36
3.4.1 Communication Protocols 36
3.4.2 The taxi ride Authentication Subsystem 39
3.4.3 The Node.js Server design (software design) 46
3.4.4 The database design (Software design) 49
3.4.5 The Front end 51
3.5 Design Based on system Functionalities 52
3.6 Conclusion 66
4 Results and Discussion 67
4.1 Introduction 67
4.2 Admin registration 67
4.3 Admin login 68
4.4 Driver registration 68
4.5 Capturing details at the beginning of the trip 70
4.6 Capturing details at the end of the trip 71
4.7 Testing the Panic Mode 72
5 Conclusions and Recommendations 73
5.1 Introduction 73
5.2 Discussion 73
5.3 Limitations and Challenges 73
5.4 Suggestions for further improvement 73
5.5 Conclusion 74
6 References 75
7 Appendix 79
Appendix A: Fingerprint code - initializing the fingerprint sensor 79
Appendix B: GPS 80
Appendix C: Server.js 82
Appendix D: App.js 82
Appendix E: MQTT Node server client code 83
Appendix F: Creating database models 86

Appendix G: Web application design 87
Appendix H: Driver Registration code..... 89
Appendix I: Fingerprint matching algorithm..... 90

Figure 2.1- loops, whorls and arches [38]..... 24
Figure 2.2: the AS608 Fingerprint sensor [39] 24
Figure 2.3 IOT devices [43]..... 26
Figure 2.4 MQTT protocol 27
Figure 2.5 Raspberry Pi 4 model B [46]. 28
Figure 2.6-raspberry pi 4 header pins diagram [47]. 28
Figure 2.7-Connection for UART protocol 30
Figure 2.8-I2C Connection 30
Figure 2.9-mongodb structure [48] 32
Figure 3.1-overall system architecture..... 35
Figure 3.2-HTTP protocol 36
Figure 3.3 architecture of the mqtt protocol 38
Figure 3.4-hivemq broker configurations 38
Figure 3.5-Taxi ride authentication sub system block diagram 39
Figure 3.6-fingerprint scanner circuit diagrams..... 40
Figure 3.7-Panic buttons circuit..... 42
Figure 3.8-seat occupancy sensor circuit diagram 43
Figure 3.9 connection of GPS sensor..... 45
Figure 3.10 flow chart of the server design 47
Figure 3.11-system use case diagram 52
Figure 3.12-admin registration..... 53
Figure 3.13-login for the administrator..... 54
Figure 3.14-driver registration design flow 56
Figure 3.15 driver authentication..... 58
Figure 3.16-trip beginning program design 59
Figure 3.17-trip end program design 61
Figure 3.18 raising a panic..... 63
Figure 3.19-resolving a panic event..... 64
Figure 3.20-handling a crime..... 65
Figure 4.1 admin registration form being filled..... 67
Figure 4.2 admin registration details in the database..... 67
Figure 4.3message on successful registration..... 68
Figure 4.4 login with wrong password 68
Figure 4.5 driver registration form..... 69
Figure 4.6 driver registration server side, with 200 ok status..... 69
Figure 4.7 driver's details in the database 69
Figure 4.8 driver's dashboard with the HMI and the driver's fingerprint 70
Figure 4.9 passenger's fingerprint details received on the node.js server..... 70
Figure 4.10 passenger details in the database upon riding l 71
Figure 4.11 arrival details successfully sent to the database. 71
Figure 4.12 Panic mode 72
Figure 4.13 message sent on panic 72

1 INTRODUCTION

1.1 Introduction

This chapter provides an introduction to the project topic, the chapter discusses the project background, problem statement, objectives and the project scope.

1.2 Background

Taxis play an important part in the transport industry worldwide. Taxis increases mobility, they allow for prompt pick-up and drop off, they are convenient and offer a 24/7 service, they are very helpful to people who have disabilities and the elders and they are relatively cheap [1]. The taxi industry provides employment to a large number of people and most of them are self-employed. Closely related to the taxi transportation system is the rise of the ride-sharing services which are offered by companies such as Uber, LYFT and via. In ride-sharing services, drivers connect with passengers via a website or mobile application and drivers carry passengers to their destination for a fee which they may receive electronically.

In Zimbabwe the public transport sector comprises of publicly owned transport buses and privately owned mini-buses, and taxis. ZUPCO (Zimbabwe United Passenger Company) is the only government owned public transport company. Most people, however, use privately-owned commuter minibuses. These Minibuses don't have published schedules or routes, but are widely used. Non-published routes and schedules are learned by local commuters **[2]**. Most taxis are privately owned and are registered under different private taxi associations such as Gibela Taxis, Expricos, Centrust Travel, and City Line Taxis. Apart from these registered taxis, Zimbabwe also has seen a rise in the number of privately owned personal cars getting into the public transport industry without registering with any public transport authorities. Most of the people involved in such business are owners of Honda fit and Toyota wish. Zimbabwe has seen an introduction to the ride sharing services some years ago, but the majority of the people are reluctant to use the services due to the economic situation and some other reasons. However, there is still a certain group of people who use the services, especially foreigner and those in the upper class in the country.

Given the nature of the taxi service sector, a lot of issues pertaining to safety are rising. Taxi drivers have been reportedly hijacked, robbed and in some cases killed due to the fact that they work during the late hours and even during the day they are sometimes asked to carry people to secluded

areas and robbed in those areas. The situation affects both drivers and riders as in some cases drivers take advantage of the rider's conditions at night or when they are drunk. The introduction of ride-sharing services has seen a rise in the security concern all over the world. There have been a lot of reports of drivers being robbed or hijacked. In these incidents, the offenders would arrange a rideshare trip on their mobile application and would rob or hijack the victim during or after the trip and in most of these incidents the offenders would be armed [3]. Passengers face the same situation with the ride-share services, reports show that in a number of cases the robbers have posed as ride-sharing drivers and robbed passengers of their valuables, in some cases the passengers are sexually abused, especially females. In Zimbabwe there are a number of crimes that are taxi related, one report reads, 'A number of robberies, rapes and thefts have occurred where victims fall prey to the illegal transporters who present themselves as genuine transporters' [4].

There are number of measures that have been put in place in trying to reduce taxi related crimes. Uber introduced a system to share your location with your friends and your relatives, and a system of aliveness detection using cameras. These systems have help to reduce the crime to some extent, but in cases where the rider have their smart phone taken away or their battery dies their security is totally at risk. In Zimbabwe the Vaya Africa app includes features such as verification of the driver's vehicle number plate, letting your friends know your route and also a panic button in case of a suspicious activity [5]. The challenge is that there is a considerable number of people who own a smart phone in Zimbabwe and those who own it are reluctant to using such systems. Using software or mobile app-based solutions poses a challenge as these solutions are not applicable to taxicabs and to private car owners who might want to give someone a ride. The Zimbabwean police face a challenge in solving these cases due to the lack of criminal's identity, information and enough evidence.

This project brings a solution to the taxi related crime through capturing the driver's and the passenger's fingerprints during a ride and storing them in the registrar database. The system ensures safety for anyone who needs to ride a taxicab or use a ride share even if they don't have their smart phone. The solution includes a device (hardware) that will be used to capture and send the passenger's and the driver's fingerprint to the registrar database and the details of the start and end of the trip for each customer. This will provide enough data for the police in case of any crime.

1.3 Problem Statement

Taxis contribute a lot in the transport industry and people use them on a daily basis because they are cheap, allow for mobility and can reach almost anywhere, however there is an increase in the number of taxi related crimes and there is lack of authentication and robbery data to help in investigating the taxi related crimes. In most cases the driver or the passenger fails to identify the robbers that is if they are left alive, in some cases dead bodies are found and details of which taxi the deceased was in, are not found.

1.4 Aim

To develop a system that keeps track of the identity of the passengers and the taxi drivers during a trip by recording and uploading the driver's fingerprint identity and every passenger's fingerprint identity, the time the driver and the passenger were together and the time the driver and the passenger went separate ways, to the General Registrar database for investigations.

1.5 Objectives

- To establish the development of a system to register the taxi driver's vehicle registration number, address, Name, Id number, Driver's License number and the finger print to a central database.
- To read the passenger and drivers identity using a figure print scanner
- To send the finger print Ids to a remote server for processing and storage in the database.
- To develop a program that checks whether the driver is registered and authorized.
- To develop a system that checks that the passengers are in the taxi using weight sensor and give the number of passengers.
- To register passenger's fingerprint upon arrival at their destination and send to the server for processing.
- To develop a web application for the authorities to monitor the activities of the taxis.
- To develop a web application that can track a taxi's location in case of a robbery.

1.6 Justification

Zimbabwe has seen a rise in the number of taxi-based robberies in the past years and in some cases the passengers have been robbed by drivers and while in other cases robbers are the passengers and drivers are robbed of their cars and even killed in some cases. The Taxi safety and robbery prevention system is well suited for reducing the occurrence of such incidences. If the robbers, whether the driver or the passengers, are aware that their identity at that particular time is known to the police they would not possibly take the risk of a robbery and in case they try to, the system is designed to have a panic button that will send an alert message to the police and start real time tracking of the vehicle's location.

The system is well suited for developing countries like Zimbabwe as most people do not adapt well to the use of mobility as a service provider, some do not have mobile devices and the system doesn't need the use of a mobile device.

The system has an advantage when it comes to police investigations in case of a robbery as there is enough information in the database for the investigation. The system also has an advantage in that it will be managed by the authorities and not by the different taxi association groups or mobility service providers which might cause user data mismanagement.

Apart from the driver and passenger's safety, the authorities may use such a system to monitor violation of laws such as vehicle overloading and also the information gathered can be used for statistics purposes in the transport sector.

2 LITERATURE REVIEW

2.1 Introduction

One article related to taxi robberies say "One of the biggest barriers to understanding this problem is the lack of data specifically collected on this crime" [6]. In this chapter literature is reviewed to show that there is a security problem related to taxis, the chapter discusses the solutions already put in place and narrows down to the proposed solution.

2.2 Public transport industry

What is public transportation system?

Public transportation systems is a system that is used by the general public for their day to day transportation need. The system includes buses, light rail, subways, high-speed rails, traditional taxicabs and ride-sharing taxis. Public transport requires a fare, they generally work on a schedule and mostly they have fixed pickup points and destinations [7]. Buses have been used in the public sector for long now, they are the most common mode of transportation and they help in the movement of a large group of people per trip. Trains are one of the oldest in the public transport industry. Trains use rails and they are much less flexible than buses as they have strictly fixed stopping stations. Trains are sometimes preferred to road transportation due to the heavy traffic during peak hours [8]. Trains are considered slow in general but with the advancement in technology, developed countries have managed to have very fast bullet trains that can reach speeds of up to 320km/hr. A tramcar is another mode of transport that is used mainly in developed countries. A tramcar is a rail vehicle than runs on a track built on city streets. They are named differently in different cities. In some cities they call them the trolley, trolley car or streetcar. Tramcars use electricity and are much more used in large cities where there is a high demand for transportation. Their main advantage is that they carry a lot of passengers at once, they have a higher capacity than buses. Tramcars can carry up to 512 passengers at once. Another mode of transport commonly used in developed countries is the Metro systems [9]. The metro system is commonly found in large cities such as New York, London and so on and they have the highest performance, speed, capacity and reliability compared to other modes. They are much more

efficient because they operate underground, meaning they do not operate with the normal traffic flow. [10]The majority of people rely on taxis on their day to day lives. Research shows that about 60% of the elderly cannot drive or own a car and thus heavily depend in taxi [1].

2.2.1 Taxis

The challenge faced by the modes of transport discussed so far is that they have fixed stopping areas and thus a number of users who want to get to a specific place cannot use them. The elderly and the disabled face challenges with using buses and trains and thus they mostly use taxis [1]. The Taxi fee is nearly the same as the bus fee (considering that ride sharing fee should be 29% higher than the current fee) [11] , they have the advantage of being able to reach to some places that buses and trains cannot reach. The term taxi in this paper will be used to refer to any mode of transport that carries less than ten passengers and can reach any destination. Taxis are mainly divided into two, there is the traditional taxis and ride-sharing or ride-sourcing taxis, both of these are further discussed in the following literature.

2.2.1.1 Traditional Taxis

Taxis have been around since 1897 and they were introduced by Gottlieb Daimler in German. The taxi was powered using gasoline and equipped with an equipment called the taximeter (hence the name taxi) which was used to measure time and distance and used to accurately calculate the tax fare [12]. The taxis continued to develop and nowadays we see the use of four passenger cars being used as taxis. Traditional taxi system provide point to point transportation and they calculate the fee according to zones. The taxis are regulated by the authorities, the local municipality. Traditional taxis are more flexible than buses and trains, they have a rather flexible route and they can be found almost everywhere in the city center.

2.2.1.2 Ride sharing / Ride-sourcing

With the rate at which technology is evolving, the taxi services have seen a change to the way taxis operate. The easy use of cellphones has enabled easy access of a lot of services through mobile phones, people now have access to ordering food, ambulance system and a lot more through their mobile phones. Taxi systems have also managed to integrate their services with mobile devices. Ridesharing began during World War II in 1942 when the United States government required

ridesharing arrangements in workplaces when no other transportation options were available [13]. The world also saw another period of ridesharing in 1970 during the oil crisis and when the gasoline price went high [14]. Ride sharing in the present day was triggered by the development of tracking technology (GPS) and also the easy use of smartphones [14]. Ridesharing is part of the sharing economy, which is a system that involves sharing of resources in a collaborative way that enables profits to be made from underutilized resources [15]. Sharing Economy companies include Spacer, which deals with people who want to create income from the extra space (unutilized resource) they have, Airbnb that allows people to change their home into cheap accommodation for foreigners [16]. Uber is one of the biggest companies in economy sharing, Uber matches drivers with passengers through their mobile application [16]. From what has been mentioned we see that economy sharing helps in utilizing unutilized resources and they make the cost of goods and resources cheaper by the removal of a middle man.

Ridesharing works by connecting drivers with passengers who want to move from point A to B. The driver specifies where they are going and what time and the passenger/ customer uses their smart phone to request for a ride, they check through the list of trips and book a ride, specifying the time and the location [17]. The customer also specifies how much they want to pay and they can pay on the app or in cash when they get into the car. The system uses a GPS to run algorithms that calculate the best route for the driver and also help in determining the location.

Uber Technologies was founded in 2009 and is the largest ridesharing company in the world Uber operates in 80 countries as of now and has completed more than 5 billion rides worldwide [17]. and this shows how people greatly depend on uber. Uber Technologies uses a rider app called “uber” to request a ride. The app sends a message to all nearby drivers, and when a driver accepts the ride request, the application calculates the arrival time of the driver. The application also helps in letting the riders know the information about the driver who is about to pick you up. The riders use the app to enter their destination and upon arrival, the fare is automatically charged using the payment method that you use in your uber application. Uber lets the rider rate drivers from 1 to 5 stars and this helps drivers to gain more customers easily if they have high ratings [17]. There are many other ridesharing technology companies that have risen all over the world, these include lyft, Juno, Gett, Wingz and so on.

It is evident that the taxi industry is huge and has a big impact on the community. The industry is currently being dominated by ride sharing technology [17]. Both the traditional taxis and the ride sharing system share some of these common characteristics. Most of the drivers are self-employed owner drivers, most of them do not have any academic skills, they work for long hours and they receive a modest amount of pay [1].

2.2.2 Transport system in Zimbabwe

Zimbabwe is a landlocked country that is located on the southern part of the African continent. The public transport in Zimbabwe is based on roads and rails and the road is the most used form of transport as 80% of the traffic and trade are transmitted by road [18]. The nation's road system is maintained and operated by a parastatal called ZINARA [19].

The transport system in Zimbabwe is composed of publicly owned buses, mini buses and taxis and privately owned cars. The Zimbabwean government owns a public transport company called the Zimbabwe United Passengers Company (ZUPCO). The company owns a number of buses that operate within cities and some do city to city trips. The company faces challenges in meeting the demand for public transport and has hence asked private buses and mini-buses owners to join the ZUPCO franchise [20]. Currently the company has a number of buses that it owns and a lot more from individuals, including the mini-buses. The company has certain protocols that the franchisees should follow and has seen some private bus owners being reluctant to join the franchise [20] and hence there is still a huge need in the transport system as large queues are seen during peak hours. The ZUPCO Company have well known schedules, routes and non-negotiable fares. Due to the increased demand for public transportation, the government has allowed the private transport operators to join in to help ease the public transport demand. These privately owned buses and mini-buses do not have schedules or strict routes and hence a number of people prefer them. The ZUPCO Company does not own any taxis and hence the taxi system in Zimbabwe is mainly dominated by privately owned companies. This most common vehicle models in Zimbabwe include the Honda Fit, Toyota Fun Cargo, Toyota Wish and the Toyota Spacio. Due to the fact that the taxi services are privately owned, there has been challenges in controlling taxi prices, regulating the taxi routes, schedules, regulating the parking areas and a lot of chaos has been seen in the city center due to drivers fighting for passengers, safety compromise so as to get customers, and terrific traffic jams especially during peak hours.

Ride sharing technology is not commonly used in Zimbabwe. There exists a number of ridesharing technologies, but the majority of the population is reluctant on using them. Vaya Africa was launched in 2018 and it operates a number of services such as Vaya Lift Driver, which is a service whereby a client uses their mobile phone to arrange for an immediate pick up or for a pickup in the scheduled time, the Vaya Shuttle-service, whereby five or more passengers are picked for a door-to-door service and the Vaya Club whereby clients share a car and split the cost [5]. There is also SmartGo, Lyft, zimrider and GTaxi which offer the ride sharing services using the same concepts that Uber uses. GTaxi however allows cab-hailing using only real taxis without allowing regular drivers to sign up for the service. With the rise of all these taxi ride sharing services in Zimbabwe, Uber has not yet found its way into Zimbabwe due to the fact that Uber depends much on cashless payment and affluence. The majority of Zimbabweans are reluctant on using these ride sharing services due to the fact that Zimbabwe is behind in terms of smart phone usage and internet connectivity. The ride sharing technology in Zimbabwe is commonly used by the upper class and foreigners who want a much secure way of traveling around.

Due to the failure of the Zimbabwean public transport to meet the transportation needs of the population, there has been a rise in the number of pirate taxis in Zimbabwe, especially in the routes that are not serviced by the public transport and most people rely on these pirate taxis for transport though there bring about a lot of problems in the transport industry [21].

2.3 Safety issues in transport

More than 3000 people reported sexual assaults related to Uber riders in the year of 2020 in the United States. An average of eight reports are made on violence and safety. The rate at which drivers and the riders report the crime is nearly the same which means drivers and riders face the same challenge [22]. DRC has reported an increase in robberies by criminals posing as taxi drivers or customers. The robbers often threaten to use violence in order to persuade their victims to comply, most of these are active at night. Reports show that as Uber trips increase so are the number of sexual assaults, these assaults include touching, kissing and penetration in some cases [23]. The research conducted in Nairobi in Kenya revealed that the risk associated with uber were abduction (40.8%), murder (35.7), robbery (41.8%) and carjacking (40.8%) [23].

These robberies happen differently in different places for example in some areas drivers are in much danger than the riders while in some areas the riders are in danger. In some cases, people fall prey to the robbers after they fail to identify their ride hailing driver, as is the incident of a lady that was killed in Washington after she got into a car thinking it was an uber [24]. The issues related to safety, especially in ridesharing, seem to affect mostly women as shown by the article that seeks for gender specific services as a way to deal with safety issues affecting women during ride sharing [25].

2.3.1 Causes of crimes

Unlike public transport that have scheduled times and well-known routes, ride sharing services and traditional taxis do not have well known routes, they work mostly at night carrying people in places that are not so safe. It is evident that most of the ride sharing service around the world are more common during times when crime is very high for example uber drivers work mostly during weekend nights where they are likely carrying drunk riders in the late hours [23].

To some extent the ridesharing system has loop holes, one article quotes "Uber's process for onboarding drivers is dangerously negligent. *Neither Uber nor Lyft uses fingerprints or law enforcement to background-check their drivers*. And Uber doesn't even bother to meet with drivers in person before allowing them to ferry passengers**"** [26]**.**

Taxi Drivers are always in close contact with strangers and they usually work alone which increases chances of being robbed. On the other hand, taxi riders fall victim to taxi drivers when they get into taxis full of strangers, this is much more evident in Zimbabwe where a number of robberies have been related to taxi riders who at night get into a taxi with people who are not customers but are rather a group of criminals [27].

2.4 Safety issues in Zimbabwe

Zimbabwe does face nearly the same crime cases as the rest of the world, but there are some cases that are more experienced in Zimbabwe that the rest of the world due to the difference in transport system.

As noted earlier, the majority of Zimbabweans use pirate taxis in places where the public transport does not offer services, there is concern over the alarming rate at which reports are made on crimes related to pirate taxis in Zimbabwe. In most of these robberies, the criminals use the Honda fit as this is the most common vehicle in Zimbabwe due to its relatively low cost when buying, fuel saving capabilities and its cheap maintenance. According to a report by the police 75% of the country's robberies involved the use of a Honda fit as an accessory [28]. In one report a man named Arrison Kachingwe narrates his story on how he was robbed and thrown out of moving Honda fit [27]. Arrison Kachingwe had got into a Honda fit at night when he was going to a pub where he found four guys in the car and thought the others were customers, but along the way he was strangled, beaten up and had some of his belongings taken before he was thrown out of the moving car. These cases are so common in Zimbabwe and the statistics show that the cases increase yearly. Of target are the university students as well, there have been reports of a number of students who have been robbed in taxis at night and some even killed.

In another case in Beitbridge Mr. Chioza who uses a Honda Fit as a metered taxi was killed by a gang of 7 robbers. Mr. Chioza was hired by the gang to an unknown destination where he was stabbed in the chest with a knife and killed on the spot. His car was used to do rob Mr. Sibanda whom they stabbed in the ribs and after they demanded R30 000 from him [29]. Both taxi drivers and riders are living in fear but there is no way people will stop using pirate taxis as the public transport sector cannot meet the needs of the population.

2.5 Solutions

This section discusses the literature on the existing solutions that have been implemented in trying to reduce crime related to taxis. The literature looks at the advantages and disadvantages of all the methods that are mentioned and the solutions are discussed from the more general solutions that have been put in place around the world to the specific solutions put in place in Zimbabwe.

2.5.1 World wide solutions

The taxi related crimes have caused a lot of concern all over the globe and a number of countries have responded by coming up with solid solutions.

2.5.1.1 Existing solutions

Vehicle Equipment

Use of Partitions

In the U.S screens have been installed in some vehicles to separate passengers from drivers. Screens or partitions should be effective if they make it more difficult for robbers to carry out the type of threat most common with taxi robberies in an area [30]. For example, bullet-resistant screens (and bulletproof plates behind the driver's seat) are designed to prevent robberies with a gun when the robber is seated behind the driver. Following installation of partitions in the 90s taxi assaults and robberies decreased in Baltimore, Maryland by 56%. The use of partitions proved to reduce assaults and robberies and was adopted by a number of taxis in 1998 and the assault rates decreased by 90% [31]. While the shield or partition system seemed to work effectively during this time, it came with its disadvantages which resulted in some taxi cab not adopting it as a standard for protection, these include the fact that:

- They are expensive to install
- They do not fit well in vehicles that are not purpose-built vehicles
- They limit driver-passenger interaction (and, therefore, possibly limit tips),
- They can interfere with heating and air conditioning circulation,
- They can lead to head injuries following sudden stops where passengers have not been wearing seat belts.

Use of Cameras in the Vehicles

In places like New York City, Chicago, and Sydney, Australia drivers are permitted to have cameras in their taxis in lieu of protective shields. The cameras take pictures of passengers at a regular time period and save them in a local storage [32].

Local authorities in the United Kingdom have permitted the use of CCTV cameras in licensed taxis as a way to protect the drivers and their valuable passengers. The cameras continuously record all the activities in the taxi and this has caused a lot of privacy issues as taxi drivers complain about their privacy.

The problem with this solution is that most passengers don't like cameras, there are high chances that passenger's privacy might be violated as the images are stored locally, the cameras might fail when someone hides their face while in the cab.One issue with collecting such information is that of data protection. The system has to be in control of authorities who have the ability to follow Data protection law. This solution hence doesn't allow everyone to use it, the use of such a system should be centralized to the local authorities.

GPS devices

The GPS is used by vehicles to track the location of the taxi cabs. Tracking the location of a taxi cab helps in identifying the area or route that the driver used in case of any incident. This system on its own is not enough of a security feature, it works well when coupled with other systems such as the camera or the shield. The GPS is used mainly by ride sharing services in identifying the location of their vehicles for use in organizing trips [33].

Alert systems

Alert systems come is a number of ways but most safety alert systems make use of panic buttons to alert the authorities of first responders of any incidence in the taxi. This system is good safety feature when combined with other systems such GPS tracking upon alert [33].

Uber Safety measures

Uber has recorded many crimes related to taxis and this has led the company to consider improving its safety features, these features are discussed here.

Uber added a feature that enables riders to send a text message to local authorities with one tap that automatically populates trip details such as the make, model and license plate of their driver's car and their location information at the time they send the text, along with the option to type their emergency [34].

The challenge with such a system is that the passenger has to have their mobile phone all the time, which is not the case in Zimbabwe. Most people in Zimbabwe cannot adapt to such a system because a number of them don't have mobile phones, those who have them might not be having power at that time and also the robbers might first of all take your phone before you get time to report.

Another feature is called Verify your Ride will give passengers a four-digit PIN to verbally provide to the driver before entering a vehicle. When their driver arrives, the PIN will pop up in the app, and the rider can read it to the driver. The driver will enter the PIN into their app. If it's correct, the rider will get a confirmation on their app, and the trip can begin [35]. This system is a good way of providing the right taxi for the customer, but in a situation where the driver's phone is stolen, someone might pretend to be the driver and kidnap the customers.

Real-time driver ID checks is a system that ask drivers to submit a selfie periodically to confirm their identity, using Microsoft technology to compare the selfie to their profile photo [35]. The technology is much helpful in identifying the driver, but if the robbers take the driver's mobile device first and forcefully take selfies of the driver, then such a system will be no longer be helpful.

Another weakness that the Uber has is that it is mostly hardware based and does not provide devices that can be used in the car. The problem with basing their operations to the mobile app is that if someone manages to get the login details of a particular driver, they can receive trips and rob customers. Again, it is easy for customers to create accounts with details that are not theirs and hence are able to attack drivers and not enough evidence is left to start an investigation on the case.

Lyft's Safety Features

Lyft is another ride sharing company like Uber, it has a number safety features which are all based on their Lyft application, and these features include:

- **Get emergency help from ADT.**
  If you ever feel uncomfortable or unsafe, you can discreetly connect with an ADT security professional right from your app. ADT can alert local authorities and share important ride details with them, like your vehicle's make and model, license plate number, and intended drop-off location [36].
- **Share your location in real time.**
  The Lyft Driver app provides real-time ride tracking so you can share your location, route, and trip status with family and friends. You can rest assured that someone who cares about you knows where you are and where you're headed [36].
- **Predicting when you need help.**

If we notice something out of the ordinary about your ride, like it has stopped too soon or for an unusual amount of time, we may reach out to see if you need help. If needed, we'll connect you with our Safety Team or directly with emergency assistance [36].

All of these features are app based, in case someone's battery dies they are in danger, they provide nearly the same safety features as Uber.

2.5.2 Solutions in Zimbabwe

In Zimbabwe there are few or no safety features put in place yet. The ride hailing services such as Vaya are not yet common to the greater population, most people are victimized while in a Honda fit or Toyota wish pirate taxis. Crimes takes time to be dealt with, there is no adequate information related to this crime.

Ride sharing in Zimbabwe is offered by Vaya and Imotacars. They have managed to include a number of security features which include doing a background check on the driver and thoroughly inspecting the vehicle of the driver before he/she may be accepted as a ridesharing driver, this is offered by Vaya. Vaya has also added a feature to know about the driver and their vehicle registration number so as to avoid getting into the wrong vehicle. This enables passengers to share their trip details with relatives and friends so that they know where they were in case, they go missing. Another feature included is the use of panic buttons through your mobile phone. The panic button enables the passenger to contact 5 people in case of an emergency [37].

2.6 System Technology

This section describes literature on the different technologies that were used in the existing systems and in the proposed system

2.6.1 Characteristics Of fingerprints:

Fingerprints consists of friction ridges, which are raised lines and furrows, which are the valleys between those lines. The fingerprint ridge patterns are usually grouped into three types which have unique variations which include the loops, the whorls and the arches [38]. Figure 2.1 shows the loops, whorls and the arches. Loops go out and curve back to themselves, they are grouped into Radial loops and Ulnar loops. whorls have circular and spiral ridges. Arches have a pattern that forms a hill [38]..

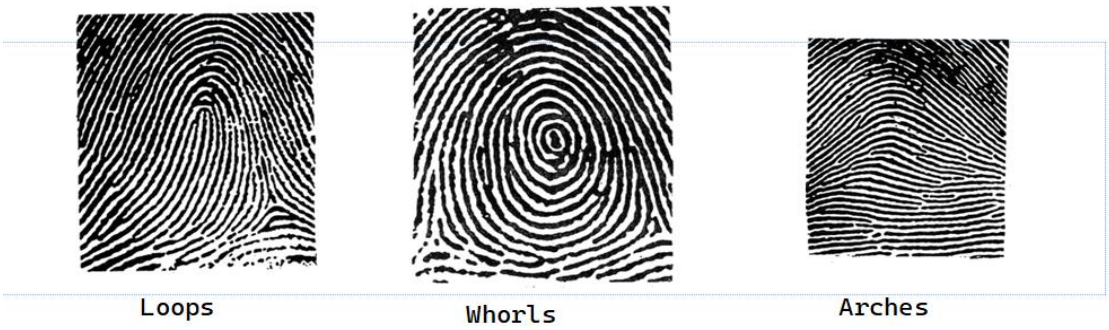


Figure 2.1- loops, whorls and arches [38].

2.6.2 Fingerprint sensors

There are three main types of fingerprint sensors, the optical scanners, the capacitive scanners and the ultrasonic scanners. Optical sensors use an LED to illuminate the fingerprint and take an image of the ridge and the valley for processing. Capacitive scanners use electricity (uses a touch capacitive surface) to identify fingerprint patterns. When a finger is placed on the sensor, the capacitive surfaces measure the charge and generates an electric charge in places where the ridges are making contact with the surface and no electrical contact is determined in areas where there are valleys. The sensor uses this information to process a pattern of ridges and patterns. These scanners consist of an ultrasonic transmitter and receiver. An ultrasonic sound is sent to the finger and some of it is absorbed while some is reflected back depending on the ridges and valleys. The fingerprint processes the received signal and if enough signal is collected a 3D rendering of the surface can be created [39].

For the project an AS806 optical sensor was used, the fingerprint sensor. Figure 2.2 shows the AS806 fingerprint sensor.

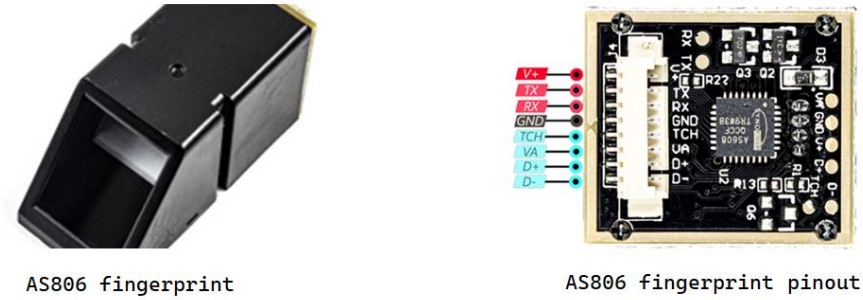


Figure 2.2: the AS608 Fingerprint sensor [39]

The AS806 has the following features

- Supply voltage: 3.3V
- Maximum current supply: 60mA
- Resolution: 500dpi
- Max fingerprint imaging time: 1s
- Storage: 162 templates
- Baud rate: 9600, 19200, 28800, 38400, 57600 (default is 57600)

The AS608 Fingerprint sensor has 4 useful pins as shown in the figure above:

- **V+**: Module power supply – 3.3V to 5V
- **TX**: Serial Transmitter
- **RX:** Serial Receiver
- **GND**: Ground

The AS806 has contains a database and a processor that run fingerprint matching algorithms (one to one and one to many matching) as well as fingerprint registration algorithms. The sensor uses UART for communicating with the microcontroller [40].

2.6.3 Internet of Things

Internet of things has to do with smart devices or sensors that can be are addressable depending on the communication technology used [41]. IOT involves a number of interconnected devices communicating with each other and a number of other services. IOT devices collect, process, analyze and store data and exchange data via a bi-directional communication between the devices. The devices use a number of communication protocols for the transmission and the reception of data [42]. Shown in the figure 2.3 is an interconnection of IOT devices.

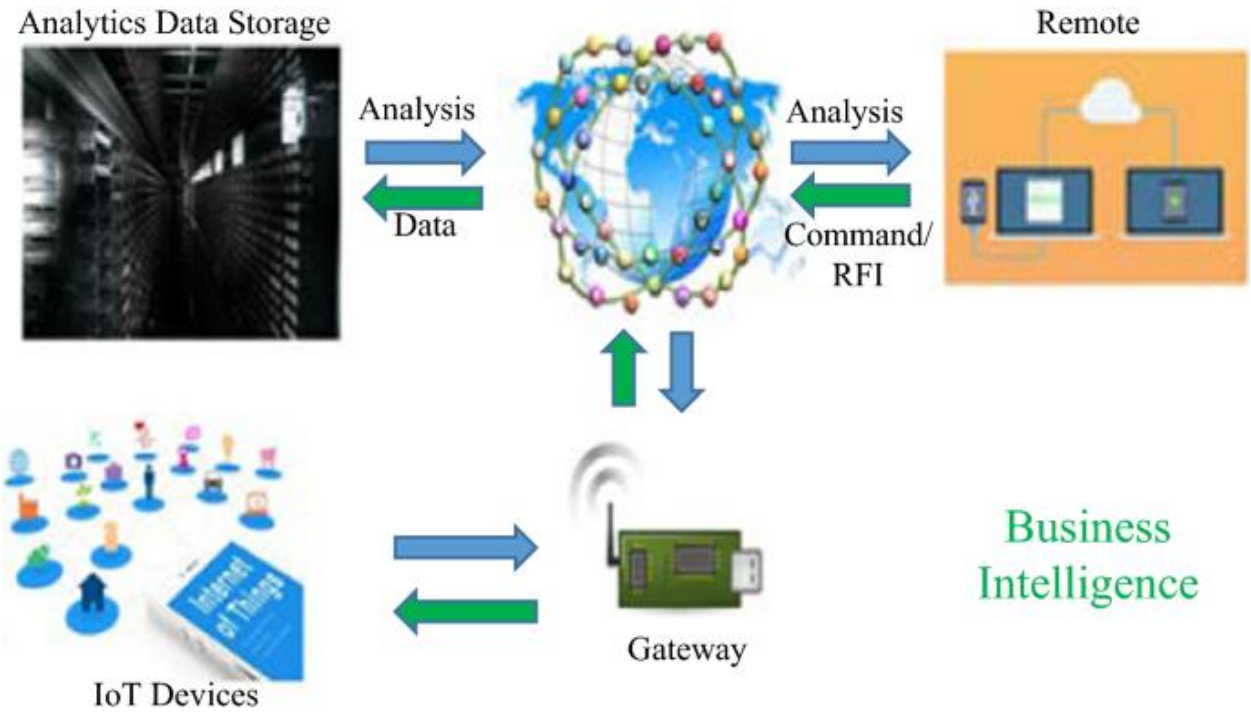


Figure 2.3 IOT devices [43]

2.6.4 MQTT

MQTT is an application layer protocol used to transmit data among devices. The figure 2.4 shows how MQTT works. The first steps involve the device sending a MQTT Connect message to the broker, the broker accepts the request and sends back an acknowledge message. The devices can subscribe to the topic. When a devise publishes a message on a topic, the broker receives the message and sends the message to all the devices subscribed to that message [44].

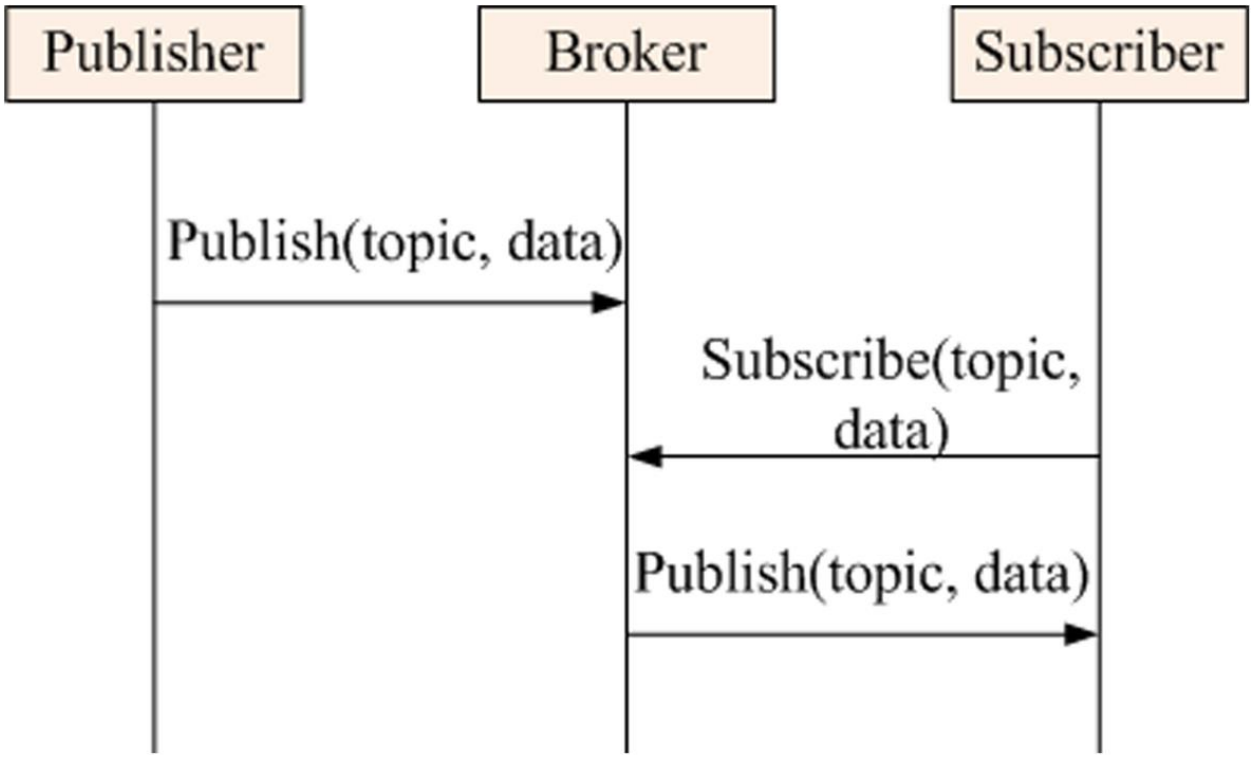


Figure 2.4 MQTT protocol

2.6.5 HTTP

HTTP is a communication protocol that was created in the early 90s. The protocol is used in the communication between the client and the server. The Protocol is based on the request response approach where the client requests resources from the server and the server responds with the resources [45].

2.6.6 Raspberry Pi 4 board

The Raspberry Pi board is a series of small single boarded computers that was developed with the goal of creating a low-cost device that could enhance programming skills and understanding of the hardware to students and tech enthusiasts. The Raspberry pi is used in many areas as a computer as well as a development board to develop projects such as weather monitoring [46]. Figure 2.5 shows the raspberry pi board.

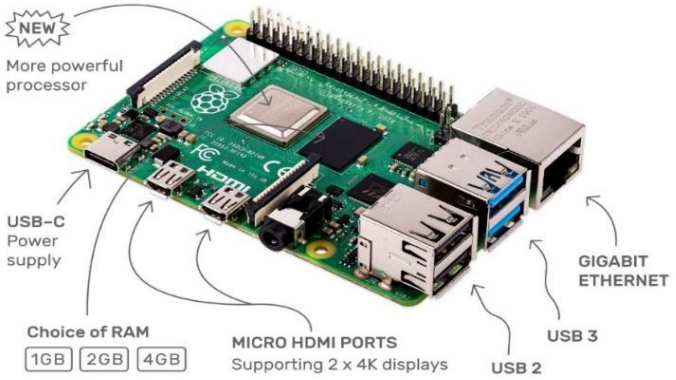


Figure 2.5 Raspberry Pi 4 model B [46].

The Board contains other features such as the gigabit Ethernet, USB 2.0 and USB 3.0 ports and a dual monitor support via HDMI type D. The board is power hungry and is powered using USB-C port 5V at 3A [47].

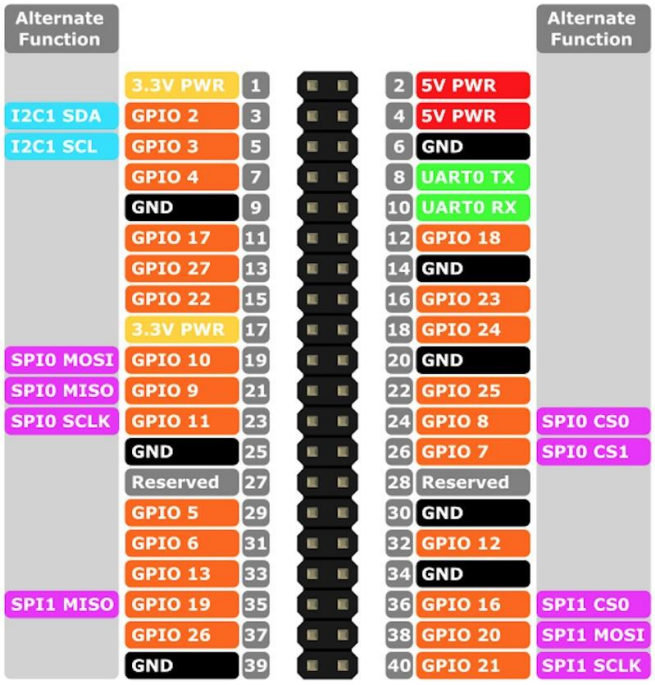


Figure 2.6-raspberry pi 4 header pins diagram [47].

The figure 2.6 shows the raspberry Pi header that has 40 pins for the raspberry Pi 4 model B. The pin numbers are shown in grey while the GPIO (general purpose Input Output Pins) are shown in orange. The pins have different functionalities as described below.

2.6.6.1 Power Pin

The raspberry Pi 4 model B has 2 power pins, the 3.3V power pins and the 5V power pins, The pins provide enough power for sensors like the fingerprint sensor and the 16x2 lcd, but the power is not enough to be used in powering motors and any component that draws a lot of current.

2.6.6.2 Ground Pins

The board has 8 pins connected to ground, that are used to provide a common reference for the different components.

2.6.6.3 Reserved Pins

The Pins (27 and 28) marked as reserved are reserved for communicating with an EEPROM.

2.6.6.4 GPIO pins

GPIO stands for general purpose input/output. These pins are used for reading (Input) and Writing (Output) data to the sensors in general. The GPIO on the raspberry pi board are all digital pins meaning they interpret a voltage as either a 1 (high) or a 0 (low). The GPIO work with 3.3V that is they output 3.3V for a binary 1 or high.

2.6.6.5 Communication Protocols using the Pins

The Raspberry Pi has a number of communication protocols that are used to transfer data in a specific way, these communication protocols use the pins on the board and some of the protocols use the GPIO and are marked as alternative in the figure above. The different protocols are as follows:

2.6.6.5.1 UART

UART stands for universal asynchronous receiver/transmitter. UART is used to transmit and receive data in serial. The UART IC is used to change parallel data from the quad core Cortex-A72 processor into serial data that is transmitted over the transmit pin of the board to the receive pin of another board which transmits that data back to parallel. The advantage of using UART is that it reduces the number of lines or pins needed to transmit the data.

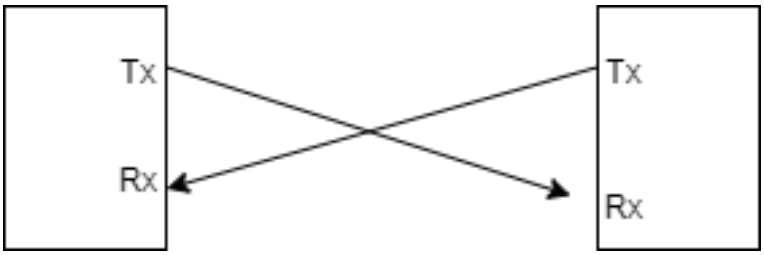


Figure 2.7-Connection for UART protocol

As shown in figure 2.7 the Transmit pin of one device is connected to the receive pin of another device and UART can be configured for Duplex and Half Duplex transmission. This protocol is used for communication between boards and also for communicating with sensors that have UART such as the Fingerprint sensor. The UART pins are marked green in the diagram above but the raspberry pi has 4 more UARTs that can be enabled and configured in the software.

I2C

Inter Integrated Circuit is a master –slave communication protocol that is used by the raspberry pi board to communicate to a number of devices. The connection is as shown below, the SDA of the master and the SDA of the slave are connected and used to transmit data while the SCL is used for the clock.

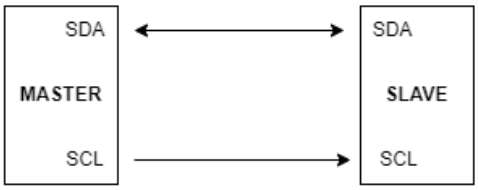


Figure 2.8-I2C Connection

The Raspberry Pi is used as the master and can have several devices or sensors (slaves) connected to the same SDA bus, the slaves are identified using their unique Ids during communication with the raspberry Pi.

SPI

SPI stands for Serial Peripheral Interface, which is an interface used for converting parallel data to serial and sending over from a microcontroller to small peripherals such as the SD cards. The SPI protocol is similar to I2C but can be configured to run faster. The pins used for SPI are marked in purple. The SPI protocol uses the following connections on the pins

- SCLK – Clock
- MOSI – Master out Slave In, used to send data from the master to the slave.
- MISO – Master In Slave Out, used to receive data from the slave
- CS - Chip Select, used to select the slave to communicate with

Just like the I2C protocol, the raspberry Pi acts as a Master with many slaves to communicate with.

2.6.7 Databases

A database is a collection of records that is managed by a database management system (DBMS). A database management system is a software package that used methods to organize and manage data. MongoDB is a non-relational, no SQL documented oriented database management system used to store large sums of data in a BSON format [48].

Mongo db has the structure shown in the figure 2.9.

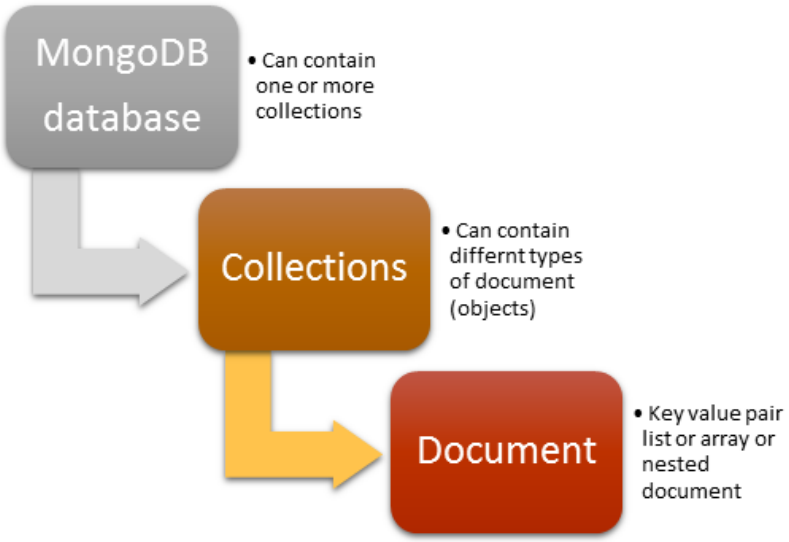


Figure 2.9-mongodb structure [48]

Features of MongoDB

- Scalability - Mongo dB uses a technique called shading to scale the data across multiple servers when the need for more storage increases.
- High Performance – MongoDB allows for fast storage and retrieval of data
- Flexibility – MongoDB can be easily integrated with a number of diverse systems [48]

2.6.8 APIs

An API is an interface that allows one program to gain access to the resource and functionalities offered by another program. Node.js is a cross-platform runtime environment. Node.js is open source and runs on Google chrome's V8 engine. It was developed to allow the use of JavaScript in creating the back end and thus it helps in executing the code outside the browser. Node.js is event driven, asynchronous, and very fast and single treaded but highly scalable. Node.js is very well suited for developing real-time applications, Data streaming Applications, JSON APIs based applications and single page applications. Node.js is used to create REST APIs.

2.6.9 Single Page Applications

A single page application is a web application that allows data to be sent between the server and the web application without refreshing the pages. The single page application breaks the web

application into a number of components that are rendered on the screen without consulting the server. React.js is used for creating single page applications that run on the browser using java script.

2.7 Conclusion

Both passengers and Customers are in danger of robbery, the only thing lacking is enough information to understand the crime. Most of the system put in place are software based and not much useful in some areas where access to mobile phones is not easy. Those that provide hardware related solutions fail to provide enough information in case of a crime. A system based on fingerprint scanning comes in to gather enough information in relation to the crime. Such a system can be integrated or built to include software solutions for those who can have access to mobile phones at the time.

3 Methodology

3.1 Introduction

This chapter dives into the methodology used for the proposed solution and also looks at the materials used in the design. The chapter uses block diagrams, circuit diagrams and flowcharts to show the interconnection between different components and to show how program algorithms were designed. The approach starts on the general design of the system, which shows how the system is broken down into subsystems and narrows down to the details involved in designing each subsystem. The chapter also describes the design of the algorithms used in designing the major system functionalities.

3.2 Materials used

Raspberry Pi Board

For the project a raspberry Pi 4 Model B was used. The model has a 1.5 GHz 64-bit quad core Cortex-A72 processor, Wi-Fi, Bluetooth 5 and 8 GB of RAM. The board was used for as the central processor for interfacing the sensors and for connecting to the internet.

Raspberry Pi 7-inch display screen

The project made use of the raspberry pi 7-inch Touch display. The full color display screen has capacitive touch sensing capabilities and was used as the human machine interface in the project design. The screen uses the adapter board DSI ribbon cable 4 to transfer touch signals and the Ground and power cables are used for powering the screen. The screen requires 5v from the raspberry pi

Fingerprint Sensor (The AS608 Optical Fingerprint)

Two AS608 fingerprint sensors were used, one for the driver and the other for the passengers.

Node.js

Node.js was used to create the REST API.

Mongo dB database

Mongo Db was used for creating and managing the database.

React.js

React.js was used in designing the Frontend.

3.3 System high level design

Figure 3.1 shows the high-level design of the system. The system is made up subsystems that communicate with each other over the internet (IoT) using different communication protocols.

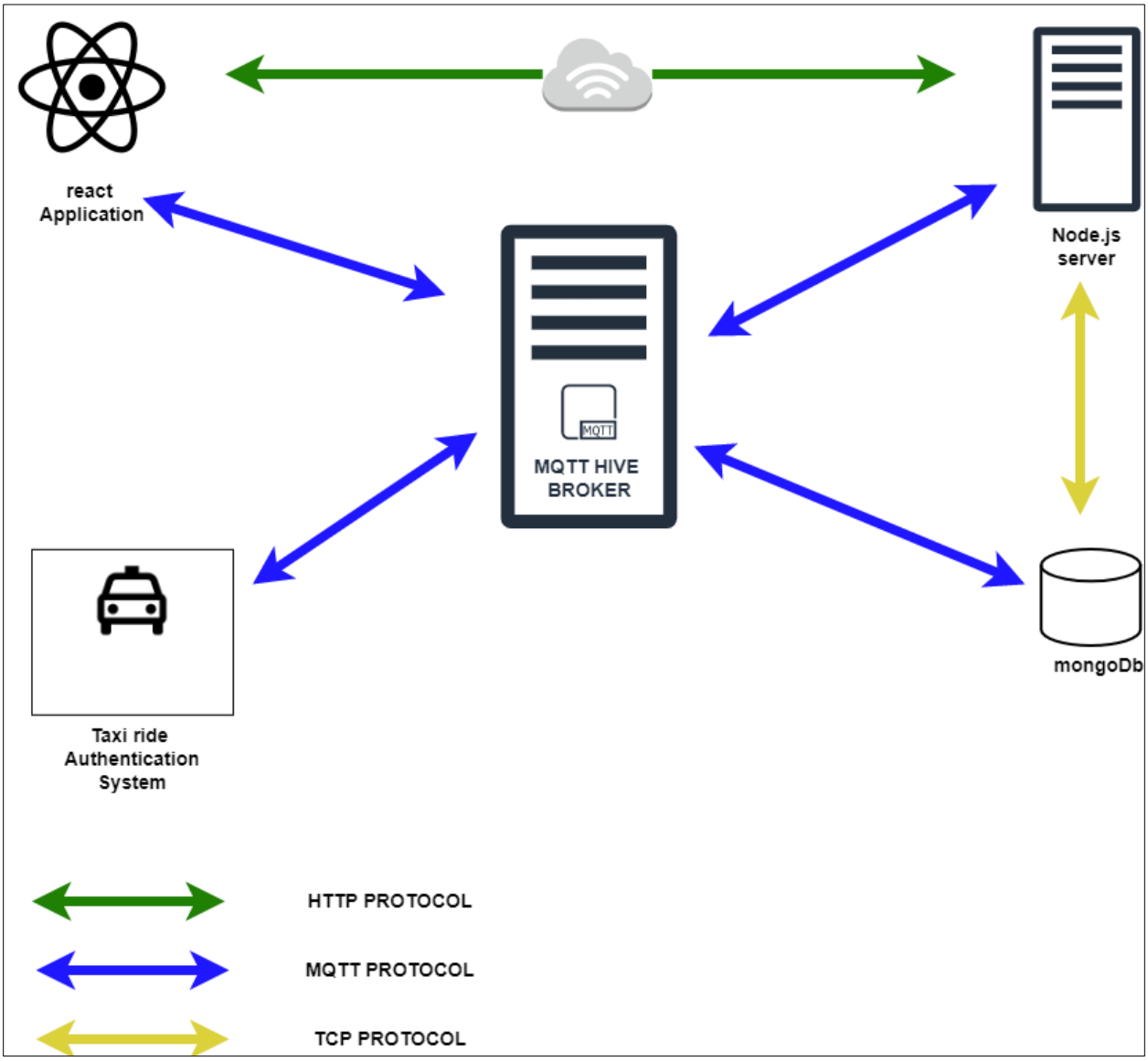


Figure 3.1-overall system architecture

As shown in figure 3.1, there are four subsystems that include the front-end (react application), the backend (node.js REST API), the database (mongo dB) and the taxi ride authentication system (the device that is installed in the taxi). The system design is based on the internet of things approach in which the subsystems are processing and exchanging data using a number of wireless communication protocols. The following topics describes the design used in each subsystem.

3.4 Design of subsystems

3.4.1 Communication Protocols

The design of the whole system was based on a number communication protocols which included MQTT and HTTP. This section describes how MQTT and HTTP were used in the design of the system and how they played a major role in subsystems real time communication (Internet of things).

3.4.1.1 HTTP

The green arrows in the figure 3.1 show the communication link that used the HTTP protocol in the system design. The HTTP protocol was used by the frontend web application to make requests to the node.js REST API. The protocol uses a request response approach, where the frontend request for services and the node.js REST API responds as shown in the figure 3.2.

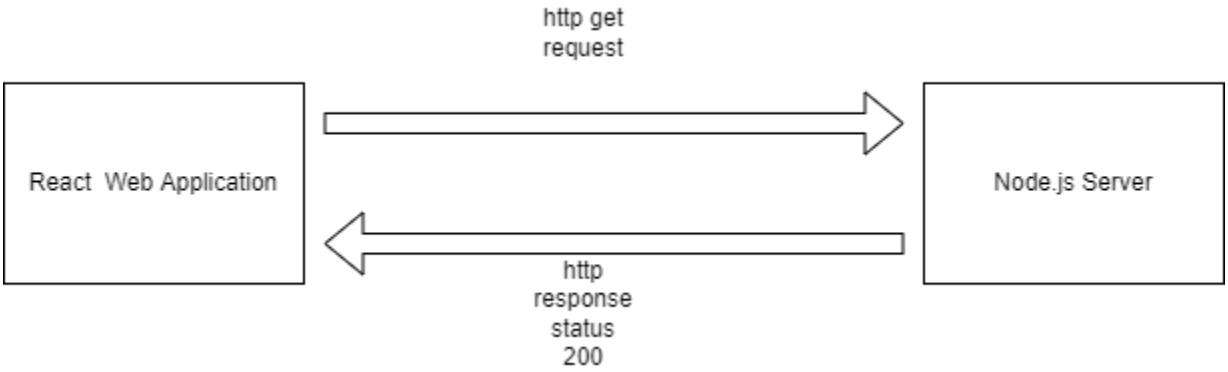


Figure 3.2-HTTP protocol

3.4.1.2 MQTT

The communication links that used MQTT are shown using the blue arrows in the system design diagram shown in figure 3.1. The protocol is used in the communication between the taxi ride authentication system, the node.js REST API and the react frontend. The subsystems subscribe to different topics and publish data to the topics as well, the following is a table of the different topics that are used in the design of this system.

Table 3.1-mqtt topics used in the system

Topic	**Publisher**	**Subscribers**	**Description**
driver/register/device_id/getFP	Frontend Application	Taxi ride authentication system	Used to get the driver's fingerprint details during driver registration
driv/register/sendFP	Taxi ride authentication system	Node REST API	Sends driver's fingerprint details to the node server
driv/register/done	Node REST API	Taxi ride authentication system and the Frontend Application	Sends a success status on adding driver's fingerprint
activity/1010/depart	Taxi ride authentication system	Node REST API	Used to send activity details on riding the taxi
activity/1010/depart/done	Node REST API	Taxi ride authentication system	Used to signal successful recording of activity on riding the taxi
activity/1010/arrival	Taxi ride authentication system	Node REST API	Used to send activity details on arriving
activity/1010/arrival/done	Node REST API	Taxi ride authentication system	Used to signal successful recording of activity on arriving
Panic/device_id	Taxi ride authentication system	Frontend Application	Used to send data during panic even
Panic/device_id/reset	Frontend Application	Taxi ride authentication system	Used to terminate panic mode

The MQTT architecture diagram is as shown in the figure 3.3. The protocol is based on the central server known as the MQTT Broker.

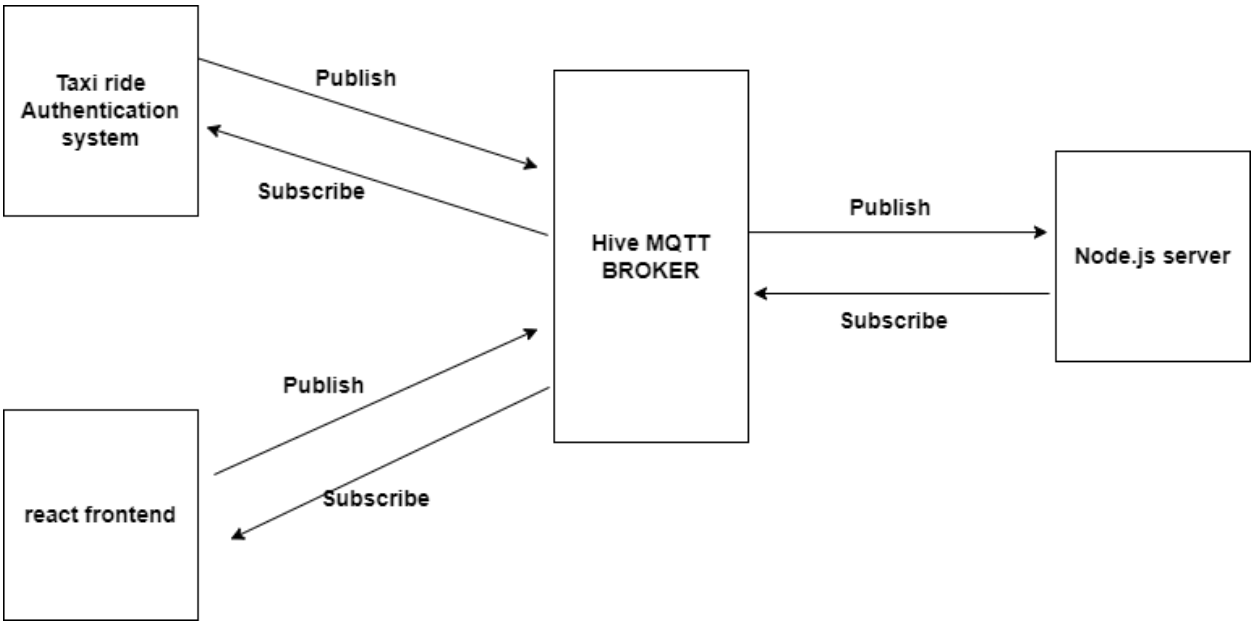
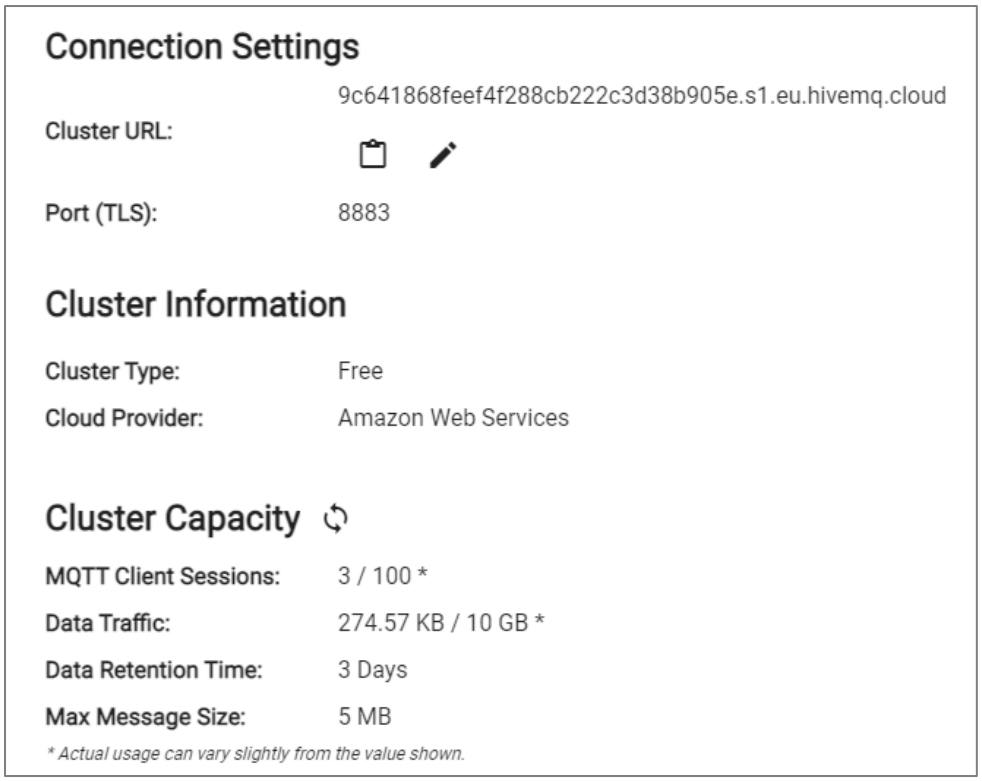


Figure 3.3 architecture of the mqtt protocol

An online MQTT broker called hiveMQ was used for the project



Connection Settings

Cluster URL:	9c641868feef4f288cb222c3d38b905e.s1.eu.hivemq.cloud
Port (TLS):	8883

Cluster Information

Cluster Type:	Free
Cloud Provider:	Amazon Web Services

Cluster Capacity

MQTT Client Sessions:	3 / 100 *
Data Traffic:	274.57 KB / 10 GB *
Data Retention Time:	3 Days
Max Message Size:	5 MB

** Actual usage can vary slightly from the value shown.*

Figure 3.4-hivemq broker configurations

As shown in figure 3.4, the broker listens to the port 8883 which is a secured port for MQTT and it has the capacity to connect 100 devices that can communicate over internet of things communication (IOT) all at once.

3.4.2 The taxi ride Authentication Subsystem

The taxi ride authentication system is the device that gets installed in the taxi. Figure 3.5 shows how the different components of the taxi ride Authentication subsystem are connected and the interfaces or communication protocols used to connect each component.

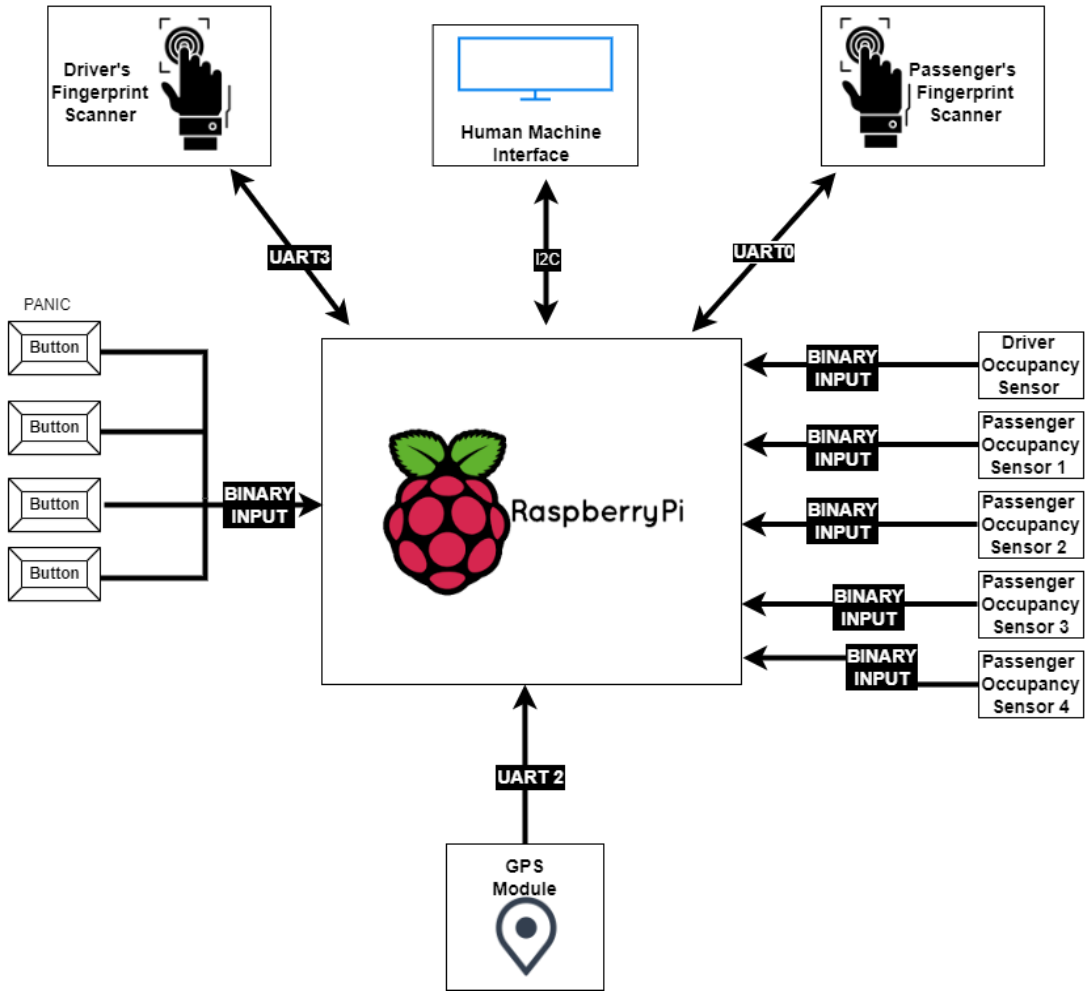


Figure 3.5-Taxi ride authentication sub system block diagram

The taxi ride authentication subsystem contains both hardware and software, the design shown in figure 3.5 is the high-level design of the taxi ride authentication subsystem, there are inputs that allow capturing of data from the outside world into the Raspberry Pi board for processing, and output devices are used to show or output the results of processing. A detailed design of each of the component and interface is as follows.

3.4.2.1 The Fingerprint Sensor (hardware design)

Figure 3.6 shows the circuit diagram of the fingerprint sensors connected to the raspberry pi header pins.

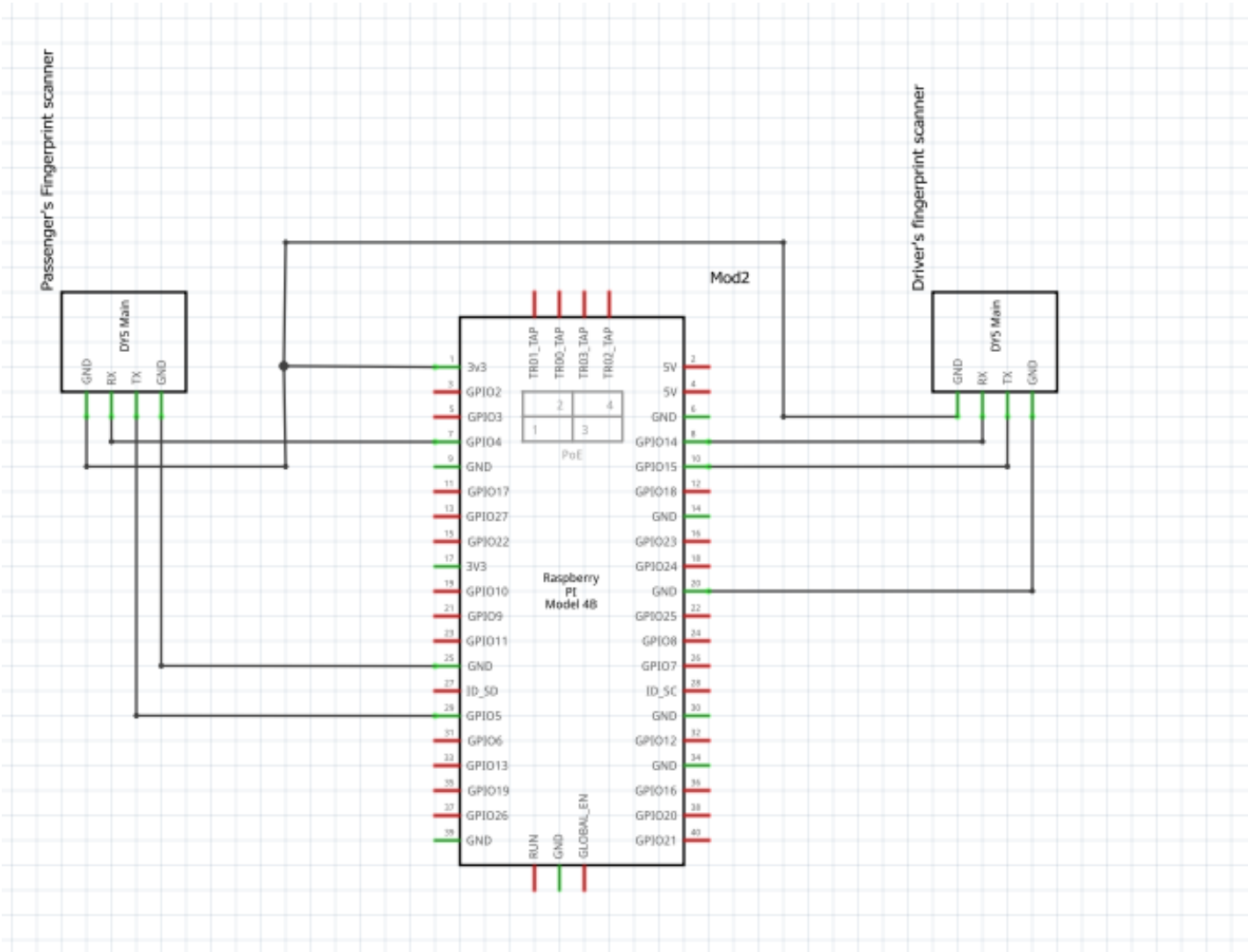


Figure 3.6-fingerprint scanner circuit diagrams

The finger print sensors use UART for communicating with the controller board. The design includes two fingerprint sensors, one for the driver and the other for the passengers. The fingerprint that the driver uses is connected to UART0 which has the transmit pin as the GPIO pin 14 on the board and the receive pin as GIO pin 15 in the raspberry pi board. The Passenger's fingerprint scanner uses UART3 which has the transmit pin as the GPIO pin 4 and the receive Pin as GPIO pin 5 on the Raspberry Pi board. The connections are as shown in the table 3.3.

Table 3.2 driver fingerprint connections

Raspberry Pi UART0	**Driver Fingerprint sensor**
GPI0 14	Rx Pin
GPIO 15	Tx Pin

Table 3.3 passenger fingerprint connection

Raspberry Pi UART0	**Driver Fingerprint sensor**
GPI0 14	Rx Pin
GPIO 15	Tx Pin

On the software side, the design involved scanning the fingerprint, generating a template, storing the template and also matching of the fingerprints. The fingerprint sensor contains a processor that runs manufacturer specific algorithms that are not exposed to the general user, but a number of functions were exposed to the user for commanding the fingerprint on the desired functionality.

3.4.2.2 Panic Buttons (hardware design)

Panic Buttons were used to signal an unusual event during a trip. Figure 3.7 is a circuit diagram showing how the panic buttons were connected to the raspberry pi board. Three buttons were connected in parallel to the GPIO pin number 17 of the raspberry pi board. The GPIO pin is pulled down to ground using the 200kΩ resistor.

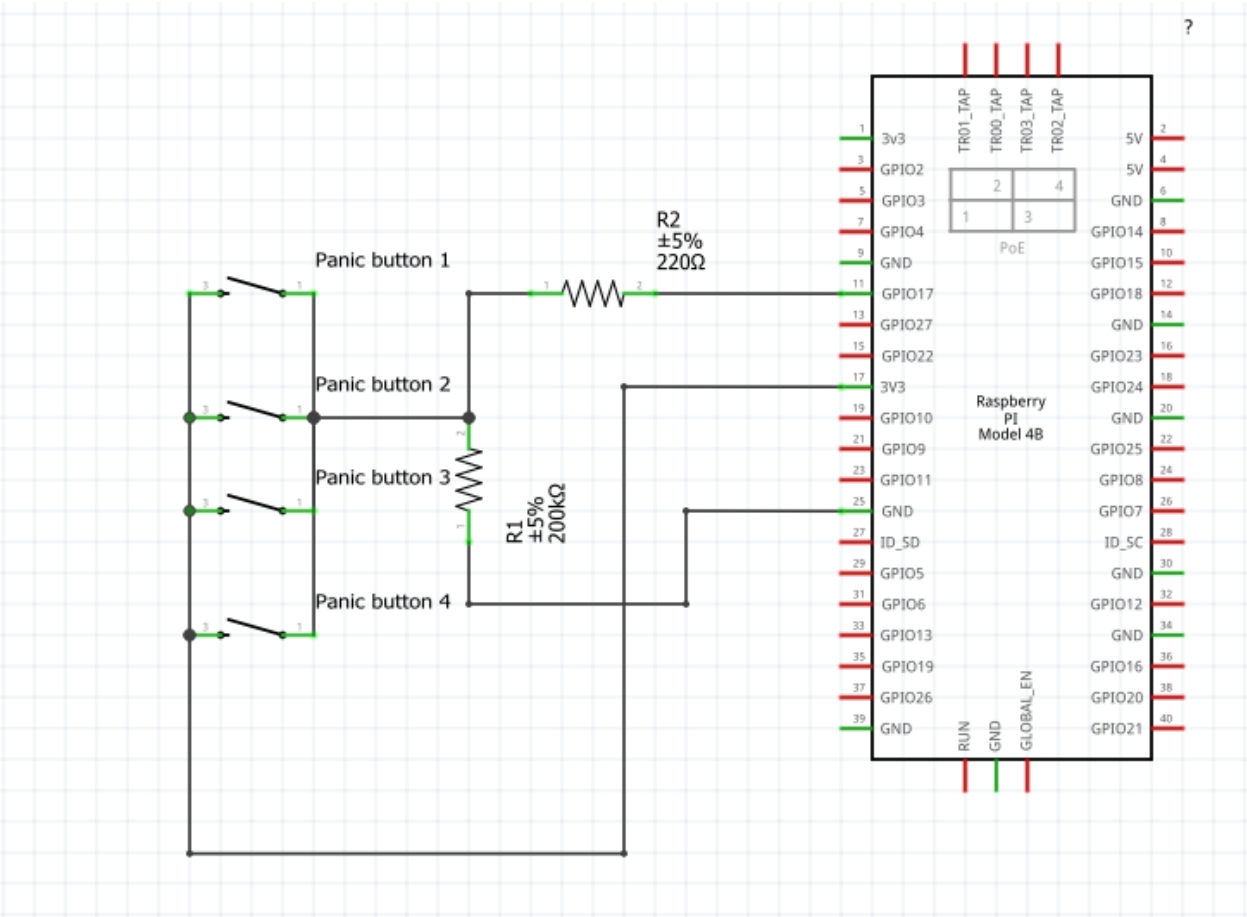


Figure 3.7-Panic buttons circuit

The buttons are implemented using normally open push buttons, when the button is not pushed the voltage read on GPIO pin17 in 0V due to the 220 ohm and the 200 kilo Ohm resistor connecting the pin to the ground. If any of the buttons is pushed, a voltage of 3.3V develops across the 200kΩ and the 220Ω resistor acts as a current limiting resistor, the GPIO pin17 reads a value of 1(high).

3.4.2.3 Seat Occupancy sensors (hardware design)

The seat occupancy sensor circuit diagram is shown in figure 3.8. The seat occupancy sensors work by completing the circuit when someone occupies the seat, hence the diagram below makes use of push buttons for the seat occupancy state. The seat occupancy sensor is simply a pressure pad that closes and opens a circuit based on the pressure put on it. The sensors are connected to GPIO pins 23, 24, 25, 17 and 22 with GPIO 23 specifically chosen for the driver's seat.

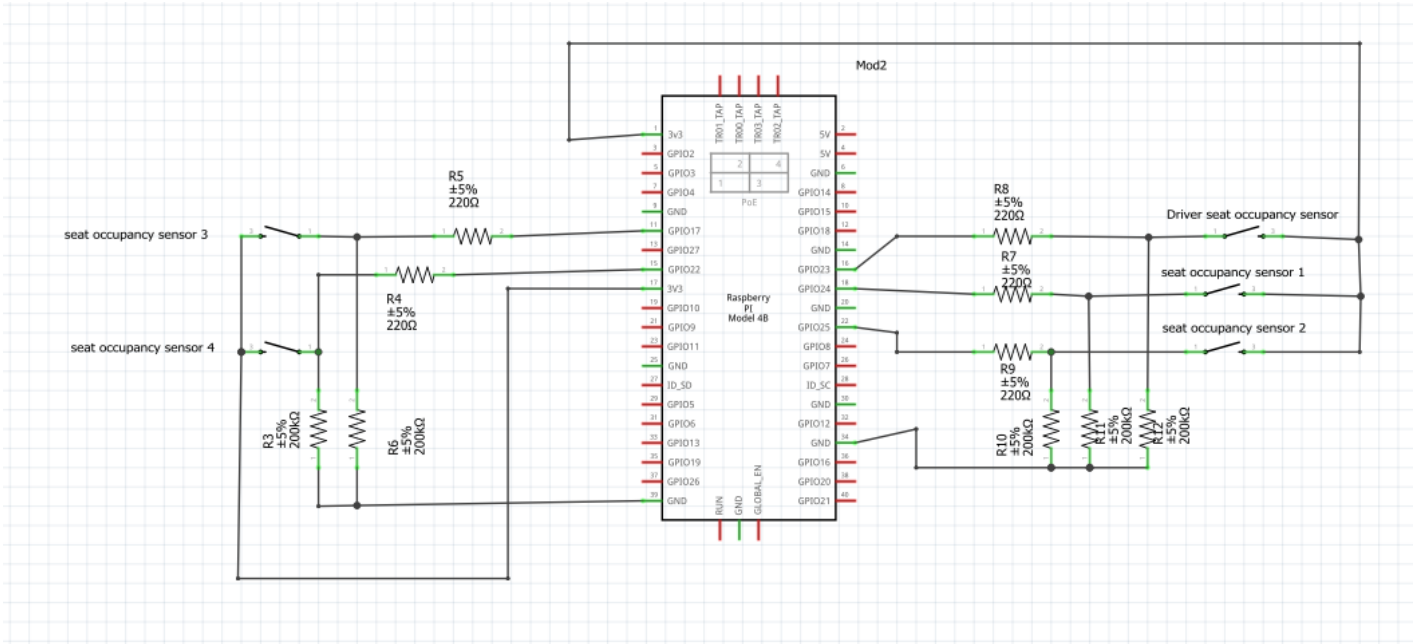


Figure 3.8-seat occupancy sensor circuit diagram

When the seat is not occupied, the sensor is an open circuit and the GPIO pin reads a voltage of 0V. When someone occupies the seat, the sensor closes the circuit giving a voltage of 3.3V across the 200 kΩ resistor which is read by the GPIO pin.

3.4.2.4 Human Machine Interface (Hardware and software design)

For the human machine interface, raspberry pi 7-inch display was used. The display uses a ribbon cable to transmit visual signals from the raspberry pi. The display has touch capabilities and it uses I2C communication protocol for communicating the touch signals to the raspberry pi.

A graphical user interface was designed using a python library called tkinter, the design was carried out as shown by the code in appendix A.

The following were the steps that were carried out in developing the graphical user interface.

Creating the window

The window is the parent container of all the graphical components that are involved in the graphical user interface of the system. The creation of the window involved calling the function **tk.Tk(),** which initialized the display window.

Creating the frames

Frames are small containers that are put inside the window so as to group together a number of components that are related. Two frames were developed inside the window, these included the bottom frame and the message frame. The function **tk.frame(window)** was used to create the frames.

Creating buttons

The bottom frame was created to house three buttons, the ride button, the arrival button and the panic button. The buttons were created by calling the function **tk.button().** The function has a number of arguments that are described below;

- Frame-to define the frame in which the button is housed.
- Text- The text to be displayed inside the button.
- Command – defines the function to be called on button push.
- Activebackground – defines the color of the background.

Configuring and rendering the window

The **window.geometry()** function was used to define the size of the window.

To render the buttons inside the window, the function **button.pack()** was called ,an argument "side=tk.RIGHT" was used to define the side on which the button is to be rendered.

The function **window.mainloop()** runs a loop that continuously monitors the changes in the components and re-renders the components. This presented a challenge as the main event loop was blocked the ever-running window loop. To solve the problem, the functions invoked on button press were run as threads. Threading allowed multiple processes to be executed at the same time without blocking. To invoke a function as a thread, the function **threading.thread()** was used.

3.4.2.5 The GPS Module connection (hardware)

Figure 3.9 shows how the connections were made. The GPS Module operates at 5v logic and uses UART for communication. To connect it to the raspberry pi, UART 2 was used and the receive

pin of the GPS module was connected to the transmit pin of the raspberry pi while the transmit pin of the GPS modules was connected to the receive pin of the raspberry pi.

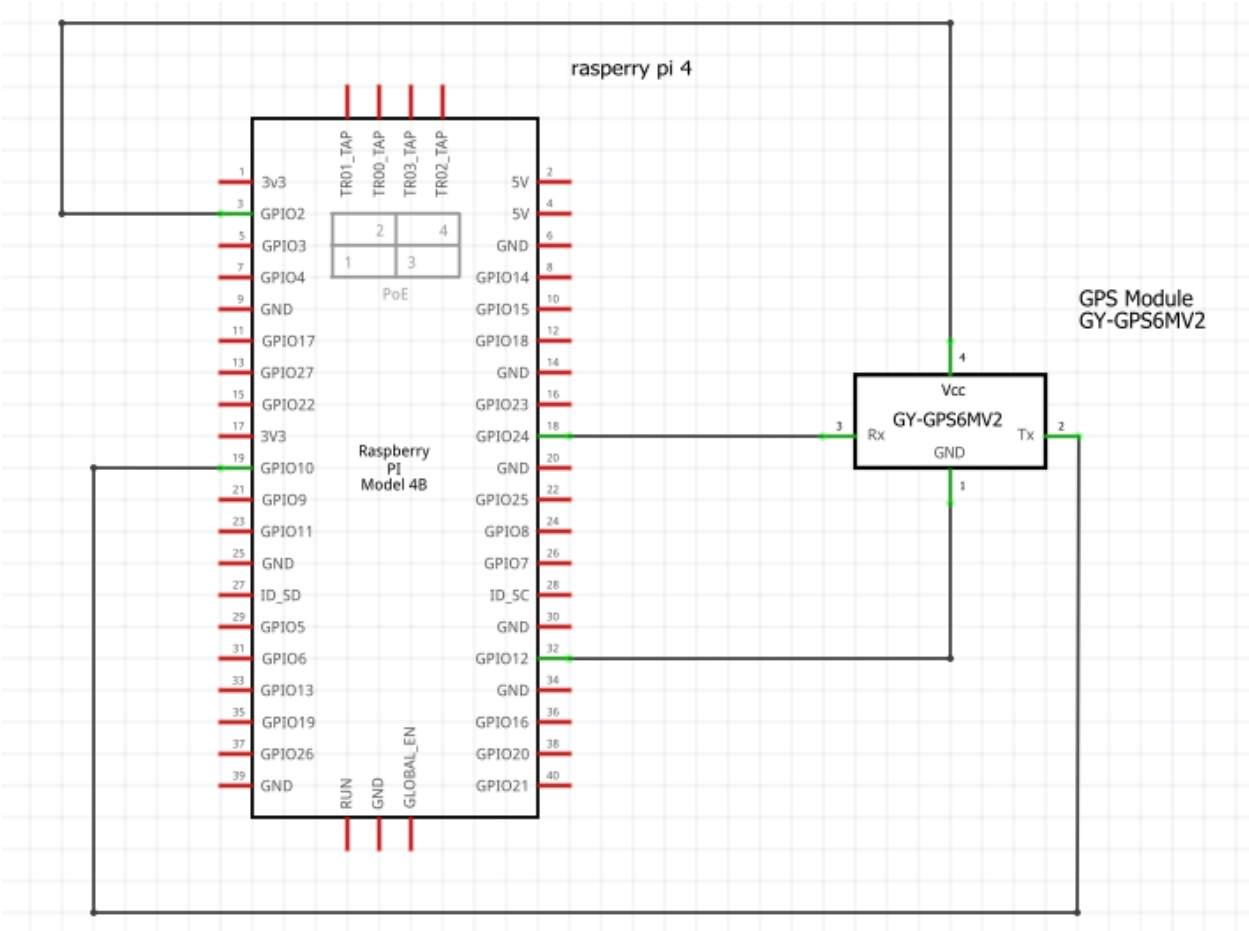


Figure 3.9 connection of GPS sensor

The GPS module was used to read the location co-ordinates and the values were gives as longitude and latitude. The code in appendix B was used for designing the program for reading the co-ordinates. The program starts in the function in line 45, the program initializes the variable to be used for storing the latitude, longitude and the buffer. The program then used a while loop to read the data from the UART serial receive pin as shown by the code in appendix B line 60. Once the data has been received, the program calls a function in line number 67 GPS_info(), which is used to convert the read bytes. The latitude and longitude are converted to degree value and the time is converted into a human readable form. After formatting the values, the program breaks out of the while loop and returns the co-ordinates.

3.4.3 The Node.js Server design (software design)

This subsystem is responsible for handling requests from the taxi ride authentication system and the React application and providing the appropriate responses. It also handles database connections, fetching, formatting and storing the data in the database. The server was implemented as a RESTful API. REST stands for Representational State Transfer. The design of the server was carried out using Node.js and express.js. A file named server.js was created to listen to continuously listen to requests and forward the requests to the app.js file. The code that was written in server.js is shown in appendix C. Line one of the code creates a process that runs in parallel with the main process listening to the MQTT subscribes topics. The main server process runs from line 3 up to line 11. The program listens to port number 5003 (line 11) and forwards the incoming requests to app.js as shown in line 9. App.js code is shown in appendix D. The program handles the requests and directs them depending on the route that is used. Line 17 handles all routes that include '/user' while line 18 handles all routes that include '/driver'. User routes and driver routes are files that are handle the final specific end points such as delete user, add user or login and logout.

The server was designed to handle 4 types of requests, which are:

- GET – used when a specific resource is required, used to retrieve data from the server
- POST- used to send or submit data to the server via HTTP
- PUT –used to replace certain resources on the server.
- DELETE- used to delete a particular resource from the server.

Figure 3.10 shows the flow diagram of how the server was designed to handle incoming request. The user.js script and the activity.js script were used to handle the final routing. The files were designed to give responses to the incoming requests and table 5 below shows the type of responses that were given by the server.

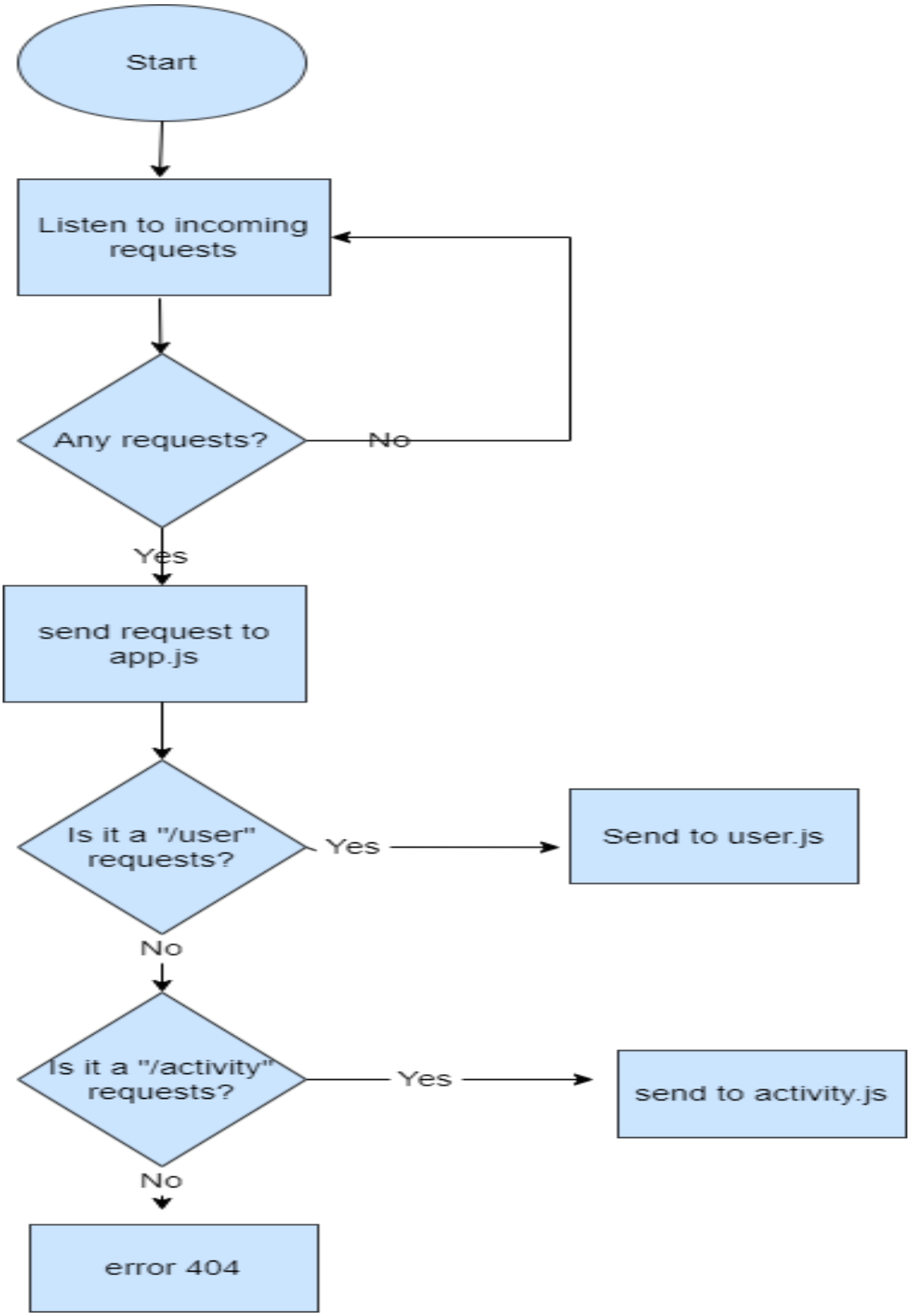


Figure 3.10 flow chart of the server design

Table 3.5 lists the end points that were used to reach to the server.

Table 3.4 http end points used

END POINT	METHOD	ACTION
/user/register	POST	Register new Administrator
/user/login	POST	Administrator Login
/user/delete	DELETE	Delete Administrator
/activity/{filter}	GET	Get activity data based on the filter (filter included trips that have panic and trips that have not been completed}
/driver/registration	POST	Register driver

A number of HTTP status codes were used to signify the type of response from the server, these status codes are as follows:

Table 3.5 - http status codes used in the server design

STATUS CODE	Description	USE
200 OK	Success	Used to signify a success during any request to the server
400	Bad Request	Used to signify that the format of the URL is not correct.
401	Unauthorized	Used when a user tries to access resources that are protected without the authority.
404	Not found	Used to notify the sender that the specified URL is out of the URLs that can be handled by the server
500	Internal Server error	Used to signal an error in the server while handling the request

Handling MQTT requests

As shown by the code in appendix C line 1, a child process was evoked which continuously monitors if there is any data from the incoming MQTT topics. The node server was designed to run as a client to the MQTT broker. The code in appendix E line 129 to 130 was used to subtribe the server to three topics. The server also published to the topics shown in line 78 and 111.

3.4.4 The database design (Software design)

The database was used for keeping all the records of the system. The database management system used was Mongodb, which is a non-relational database. Three collections were defined for the database, the administrator's collection, the driver's collection and the activities collection. Appendix F shows the scripts (schemas) that were used to create the models. The schemas shows the fields for each collection created, the database automatically generated a unique id for each record that was created.

The Administrators schema

In this schema the fields were defined as in table 3.7.

Table 3.6 the administrator field description

Field	Attributes	Description
Name	String, required	The administrators name saved a string of characters
Email	String, required, unique identifier	The email was used to uniquely identify the administrator
Password	String, required, hashed	The password was used to for authenticating the administrator during login.
Refresh token	String, required	Used to refresh sessions between the server and the react application.
Role	String, required	Three roles- Administrator, Editor and the User

The Activity schema

In this schema the fields were defined as in table 3.8

Table 3.7-activity fields description

Field	Attributes	Description
Device Id	String, required	Used for the device Id, to identify the device related to the activity entry.
PassFingerprint	String, required	Used for the fingerprint template
DepartureTime	Datetime, required	Used to show the date and time when the trip started.
ArrivalTime	Datetime, required	Used to show the date and time when the trip ended.
Departed	Boolean, required	A flag used to indicate that a trip has started (indicated by true)
Arrived	Boolean, required	A flag used to indicate that a trip has ended (indicated by true)

The Driver schema

In this schema the fields were defined as in table 3.9

Table 3.8 driver's model fields description

Field	Attributes	Description
Name	String, required	Driver’s name
Surname	String, required	Driver’s surname
Email	String, required	Driver’s email address
National_id	String, required	Driver’s national id, used as a unique identifier for the driver
Fingerprint	String, required	Driver’s fingerprint template
Phone	Number, required	Driver’s phone number

Responder	Number, required	Phone number for the responder in case of an incident
VehicleReg	String, Required	The registration number of the vehicle
DriverLicence	String, Required	The driver's licence number
Address	String, Required	The Physical address of the driver.

3.4.5 The Front end

The front end was developed using React.js. Request for different pages are not made to the server or backend but rather components were created in react.js and access to different components was handled by react router. The components represented pages in some instances and in some cases the component had a number of components in it.

3.4.5.1 The landing page

In the landing page a component was designed to house buttons that led to different pages. These buttons included the Add admin button, the log in button, the logout button, the panic review button. These were designed using html and styled with cascading style sheet.

3.4.5.2 Design of the forms

The login page, driver registration page and the admin registration page included forms that were designed using material UI. The code in appendix G shows how the forms were created and the mechanism for sending the forms to the server. The input fields were rapped around the form container a listener function used to set a global variable whenever someone changes the input field. To send data to the node.js server, axios was used to send the data. As sown in appendix G, Axios takes the user data and send it to the server, the responses are then handled by Axios.

3.5 Design Based on system Functionalities

The System was designed to have a number of users who have different uses and access to the system. This section describes how the different functions of the system were designed. The system had 3 types of users namely the driver, the passenger and the administrator. Figure 3.11 shows how the different users interact with the system.

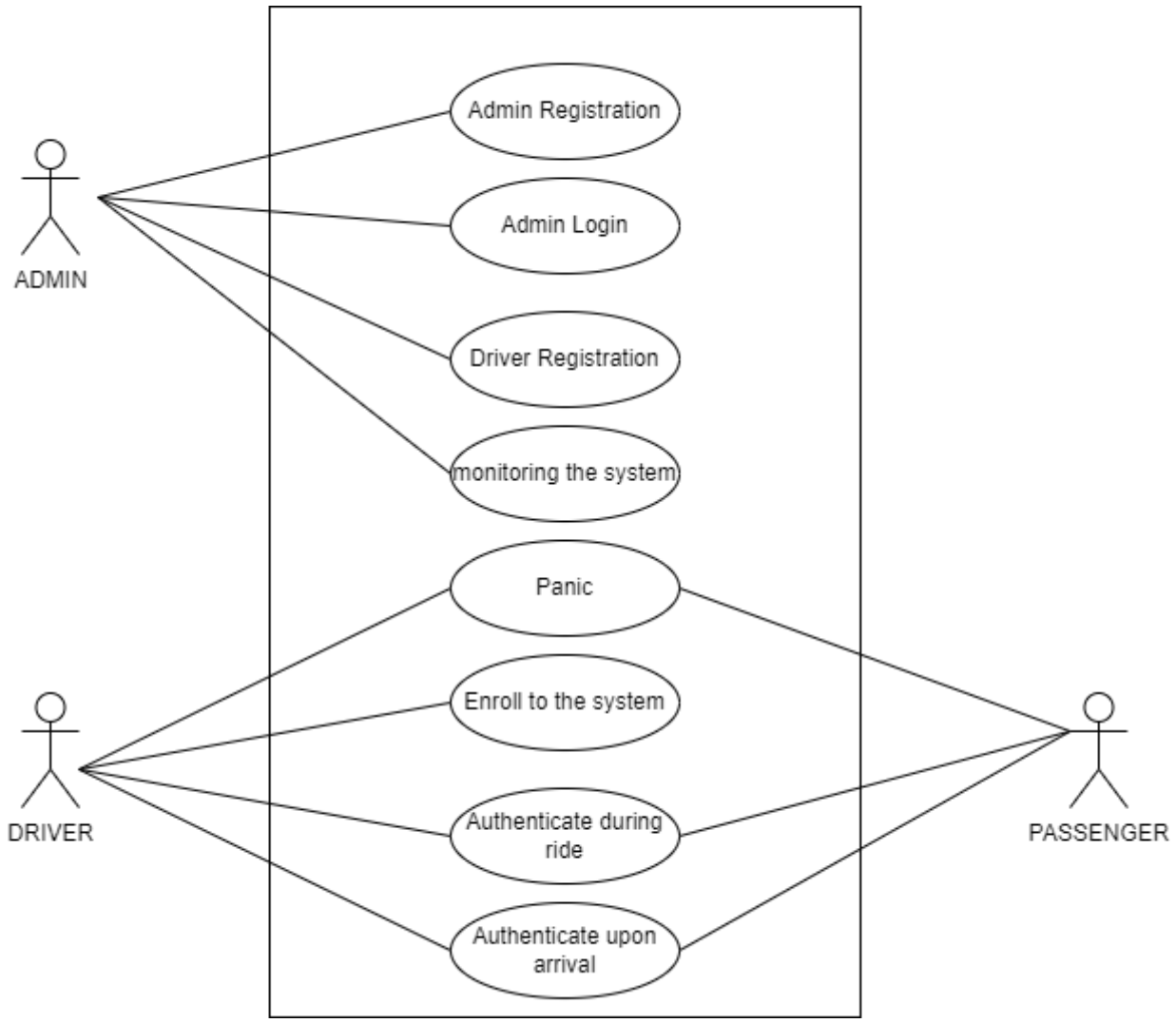


Figure 3.11-system use case diagram

The design for the tasks performed by the system is as follows:

3.5.1.1 Design of Admin Registration and login to the system

The design for the admin registration involved the interaction of two subsystems, the web application and the server (Node.js) communicating via the http protocol. Figure 3.12 shows how the design for the registration was carried out.

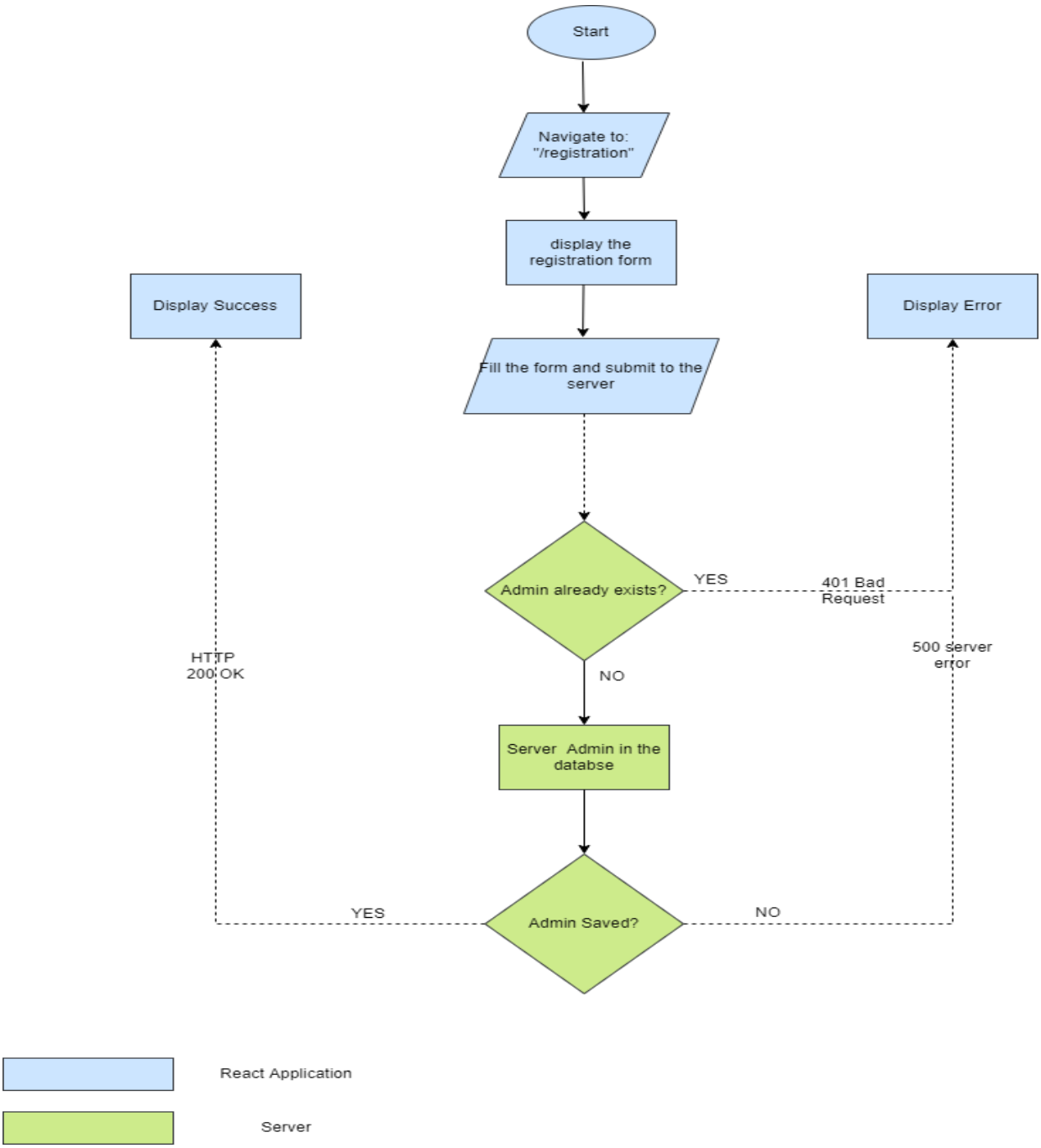


Figure 3.12-admin registration

The URL "/registration" was assigned for accessing the form that was used to register the user. The design involved capturing the admin name, email and password and submitting the details to the server using axios. The server then checks if the admin's email address is already in the database, if the email exists a HTTP error 401 is returned to the react.js application and the error message is displayed on the screen. If the email was not registered before, the server proceeds to generate a unique Id which it assigns to the admin in the database.

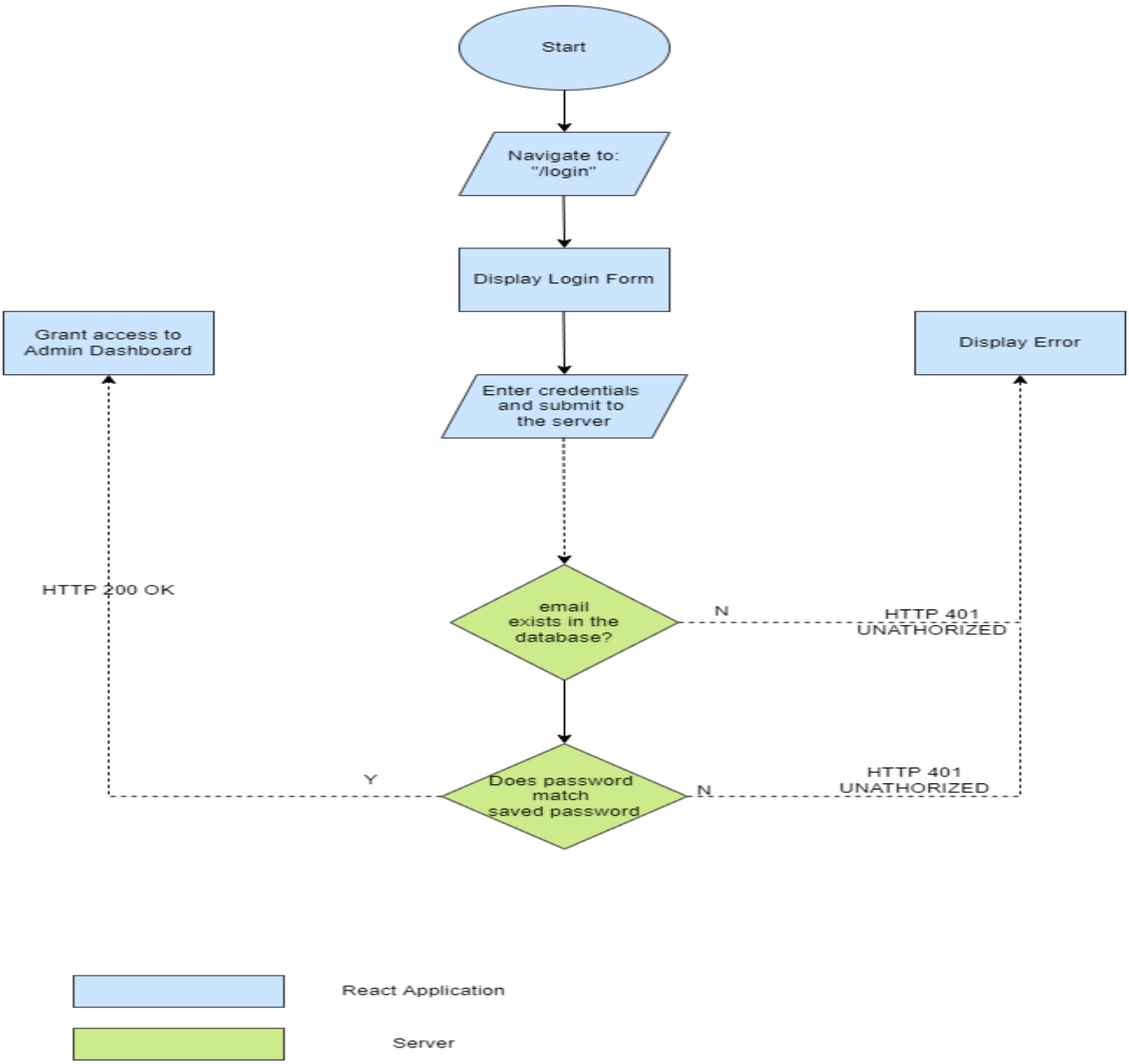


Figure 3.13-login for the administrator

Figure 3.13 shows the design for the admin login functionality. The URL "/login" was used to render the login form (component) on the react application. The form was designed to capture the email address and the password of the admin and submit it to the server using HTTP. The server receives the data and checks if the email address is registered (stored in the database). If the email address is not registered a status of 401 is sent to the react application with the message "not registered", the application receives and displays the message. If the email address is registered, the server continues to check if the supplied password matches the one saved on the database. If the passwords do not match, an HTTP status 401 is sent with an error message "/authentication failed". If the passwords match, the server returns a 200 Ok status code to the application and the user is redirected to the admin dashboard.

3.5.1.2 Driver registration

Registering the driver involves capturing the driver's details and matching them to the authentication hardware that is installed to his/her car. Driver registration is carried out by the administrator and the process flow is as shown in the figure below.

3.5.1.2.1 Capturing driver's details

The administrator navigates to the URL "/driver/registration" and a driver registration form is displayed. The details required for the driver include the Name, surname, National Id, email, phone number, Vehicle Registration number, and the phone number for the first responder. The driver's details are bundled together into a json object which is sent to the Node.js server via http. The server checks if the driver is already registered and if the driver is registered, the server responds with "400 bad request" http status code and the error message is displayed on the front end. If the driver is not registered, the server creates a record for the driver in the database and responds with "200 OK" http status code.

3.5.1.2.2 Adding the driver's fingerprint

Upon receiving the "200 Ok" http status code, the frontend app renders a form on the screen where the administrator fills the driver's national Id and the device Id and publishes the details to the topic "driver/register/deviceid/getFP" using MQTT. The code in Appendix H shows the process of registering the driver on the taxi ride authentication side. The device with the Id sent receives the details and prompts the driver to place his/her finger on the fingerprint scanner. The fingerprint scanner reads the driver's fingerprint, converts it to a template, stores it in its local database and

then transmit the template information via UART to the raspberry pi board. The raspberry pi generates a json object that has the driver's fingerprint template, the device id and the driver's details and publishes to the topic "driv/register/sendFP", shown in appendix H line 35 to 39. The sever is subscribed to the topic "driv/register/sendFP" and receives the driver's fingerprint details , driver's national Id and the device Id.

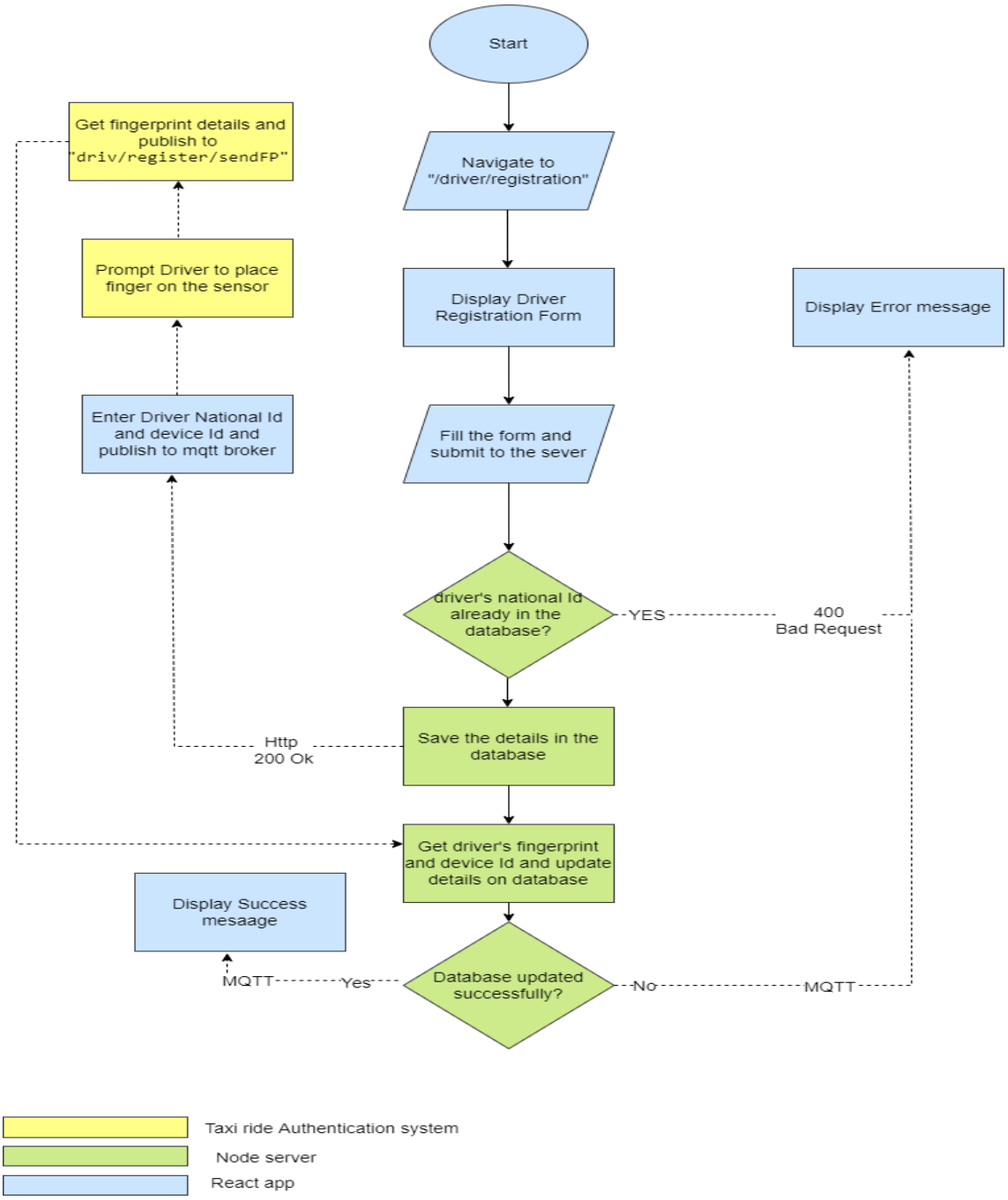


Figure 3.14-driver registration design flow

The driver's Id is used to search the driver's records in the database and update the details with the driver's fingerprint template. If the details are updated successfully, the server publishes a success to the topic "driver/registration/success" on which the front end and the taxi ride authentication system are subscribed. The front end displays a success message while the taxi ride authentication system creates a record of the driver's details in its local database.

3.5.1.3 Driver Authentication

Design of driver Authentication involved scanning the driver's fingerprint and matching it to the fingerprint that was used during registration, this process was carried out to check the identity of the driver during a trip and at the end of a trip.

Figure 3.15 shows the flowchart for the design of the program. The program first checks if the driver is on his seat, this is done by reading the seat sensors value which gives a binary value of 1 if the seat is occupied and 0 if not occupied. If the seat is not occupied, the program returns a value of zero to the calling function and sets a message on the driver's Human machine interface with the words "*driver's seat empty*". If the driver seat is occupied then the program reads the number of passenger seats that are occupied and compared the number with the number of passengers who are supposed to be in the taxi. The number of passengers who are supposed to be in the taxi is stored in the database and is incremented by one every time a passenger enrolls their fingerprint successfully. If the number of seated passengers is equal to the number of enrolled passengers, then the program scans the driver's fingerprint, runs an algorithm to compare the fingerprint to the one that is stored in the system. If the fingerprint doesn't match, the programs returns a value of 301, which signals a failure, with a message that says "*driver authentication failed*". If the fingerprint match succeeds the program returns the driver's fingerprint template id which is always 0 and the program continues to either trip ride program or arrival program.

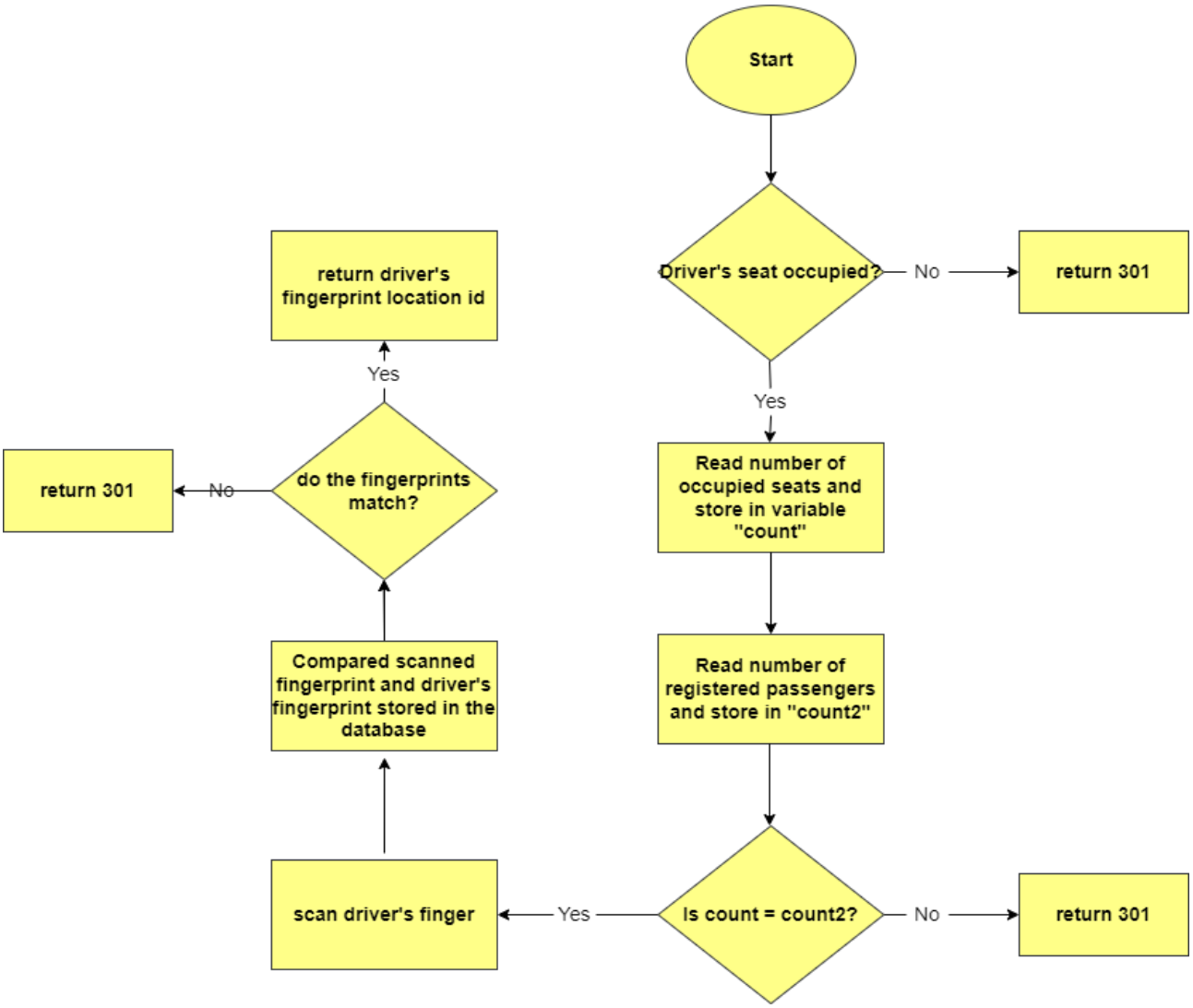


Figure 3.15 driver authentication

3.5.1.4 Ride or Trip beginning

Ride or trip beginning involved enrolling the passenger's fingerprint and sending it to the database. The program is only invoked after the driver's fingerprint details have been matched correctly. The design of the program was done as shown in the figure 3.16.

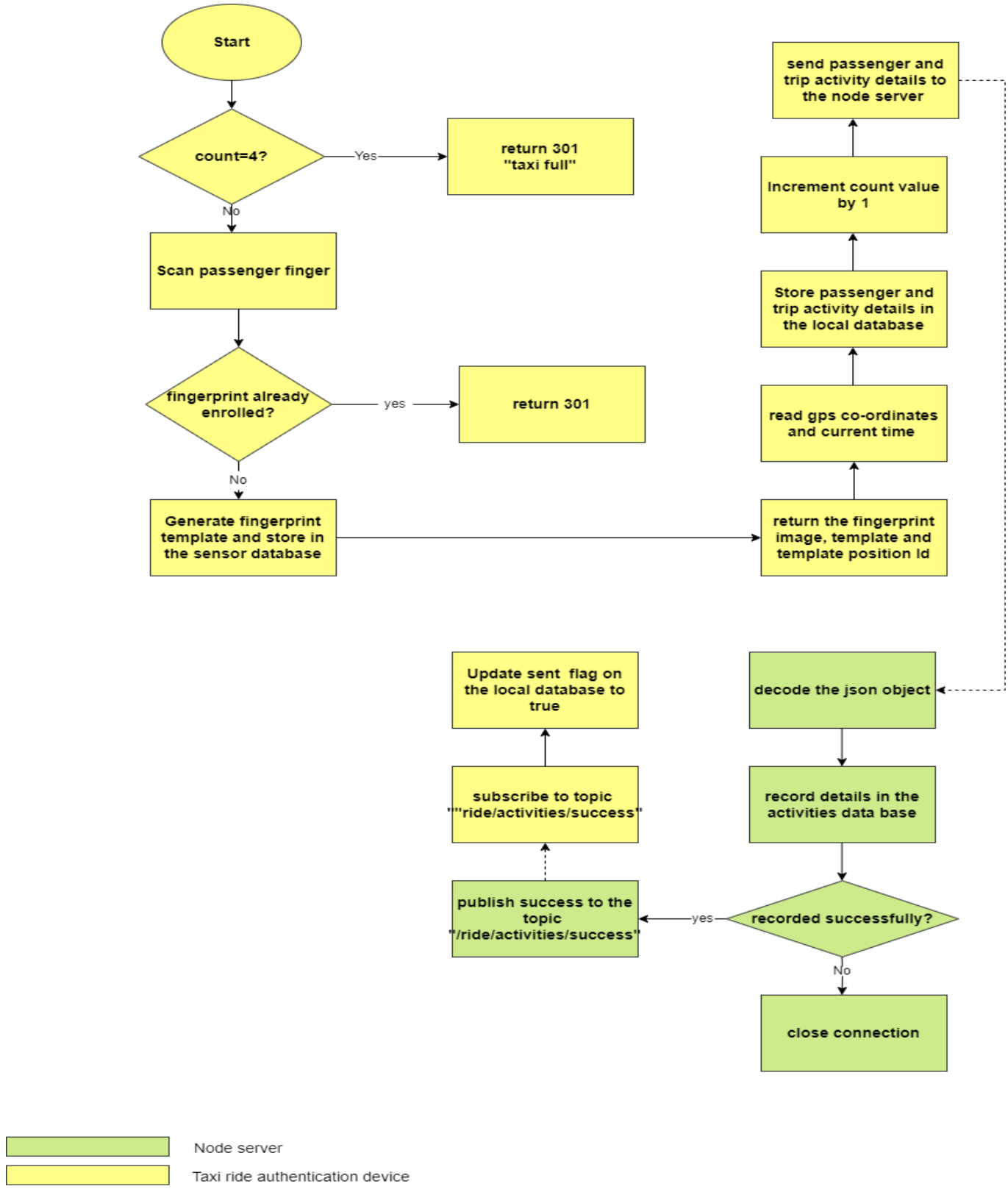


Figure 3.16-trip beginning program design

The first step is to check the number of passengers who are already enrolled in the system, if the number of passengers is more equal to 4, the program returns a value of 301 which signals failure, with a message “taxi full”. If the taxi is not full, the program asks the passenger to place their finger on the fingerprint scanner. The scanning of the fingerprint was designed to happen twice so as to generate a high-quality template. After scanning the finger, the algorithm generates a fingerprint template which is stored in the local storage of the fingerprint sensor and the fingerprint sensor returns the fingerprint template, the fingerprint template location id, and the fingerprint image.

After the fingerprint sensor has returned the fingerprint details, the program reads the location co-ordinates from the GPS module, which are given as latitude and longitude values as shown in appendix B. The program also gets the current time using the function “datetime.datetime.now” which generates the current date and time. The fingerprint details, current time and the location co-ordinates are then recorded in my SQLite database that is local to the machine. The database was designed to also include the fields “ride” and “arrived” to indicate whether the passenger trip started and ended or it only started and the passenger has not left the taxi. The “ride” field is set to 1 upon riding and the arrived is set to 0 by default. The “sent ride” and “sent arrival” were used as flags to signal whether the row of information was sent to the database online. The next step in the design was to send the data that was saved locally to the node.js server for storage in the centralized database. The data had to be converted to a json string and published to the topic “activity/1010/depart “ via MQTT.

The node server was designed to subscribe to the top “activity/010/depart” and upon receiving the data, the node.js server adds two more fields to the received data, which are “departure” and arrival. The server sets the departure field to 1 and the arrival field to zero and records the activity in mongo dB database and publishes a success message on the topic “activity/010/depart/done”. The taxi ride authentication device updates the local database activity table, ride field with the flag value set to 1 to signal that data was successfully sent to the online database.

3.5.1.5 Arrival or Trip End

The design in this section involved designing a program that runs to signal that the passenger’s trip is done and they have arrived safely. The program begins when the driver hits the arrived

button, calling a function that asks the driver to authenticate or identify himself using his fingerprint. The driver authentication is as described in the section above. If the driver identifies himself correctly, the “arrived” program runs. The “arrived” program was designed as follows.

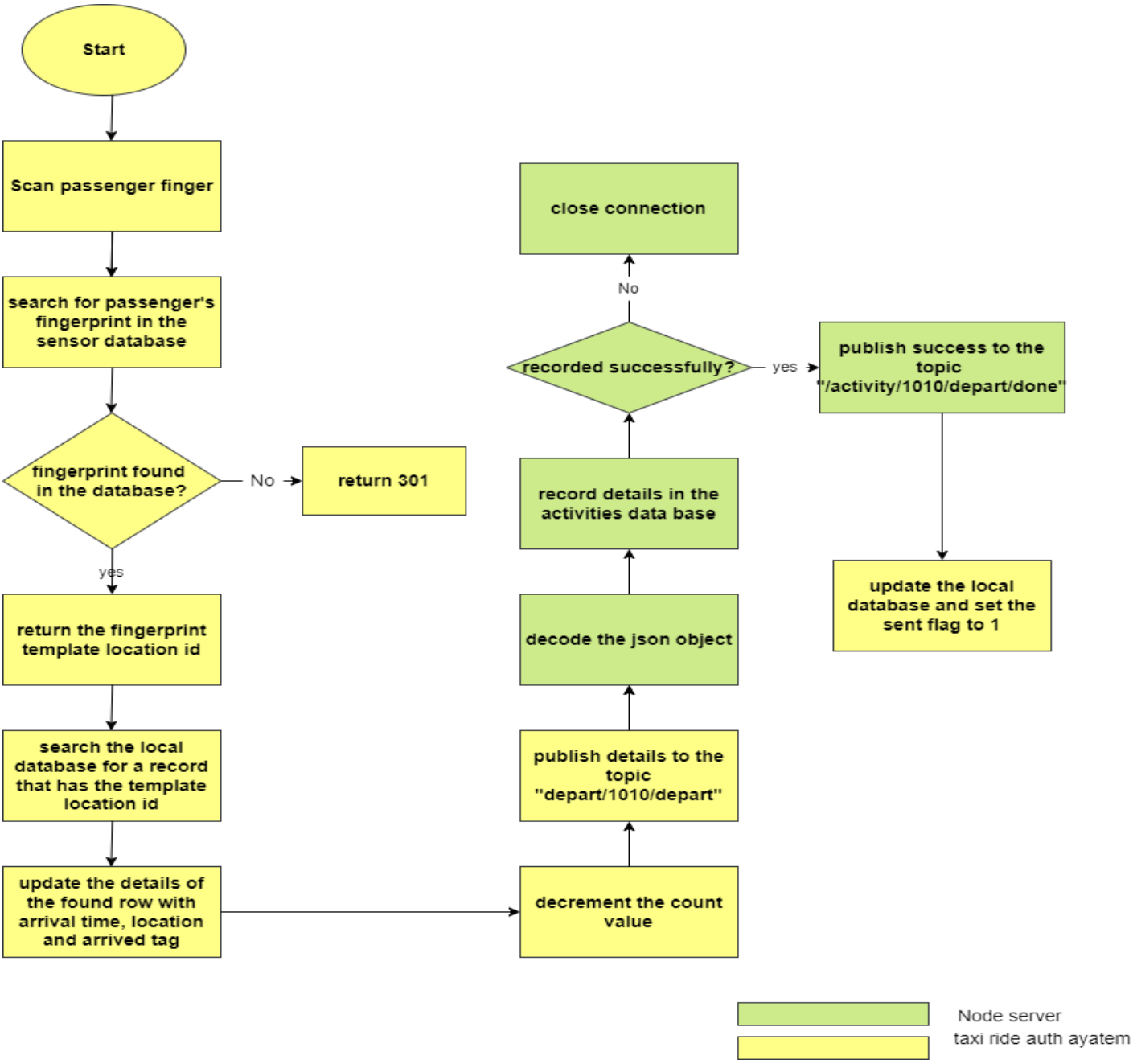


Figure 3.17-trip end program design

The program scans the passenger's fingerprint and tries to match the scanned fingerprint to the fingerprints stored in the scanner's database. If the fingerprint is not found in the database, the program throws an error and the technician or administrators have to be contacted to signal an anomaly in the taxi. If the passenger's fingerprint match in found, the location id of the template with the match is returned and used to query the local database so as to get the passengers details. If the passenger's trip details are found, the field with the arrived tag is updated with a value of 1, the arrival time and the location are also updated in the local my SQLite database.

The next step as shown in the figure 3.17 above is to send the arrival time, location and the passenger's fingerprint template id to the node.js API. This is done by publishing the details to the MQTT broker in the topic "activity/1010/arrival". The server is subscribed to this topic and upon receiving the details it decodes the results, uses the fingerprint details to filter the search results in the database. If the activity of the user is found, the server updates the arrival flag to 1 and the arrival time and location.

3.5.1.6 Panic or emergency

The panic program is a program that was designed to run when a driver or any of the passengers hits the panic button during a trip. The panic event was used to signal an event that needed an immediate attention in the taxi. The design of the event was broken down into two parts, the part designed for raising or signaling a panic and the part for resolving a panic situation.

3.5.1.6.1 Signaling a Panic Event

The design for signaling a panic situation is invoked by pushing a panic button available in the car. In the figure 3.18, the yellow color shows program steps that were designed to run in the taxi ride authentication system device installed in the taxi. The blue color shows programs that were run in the front-end application.

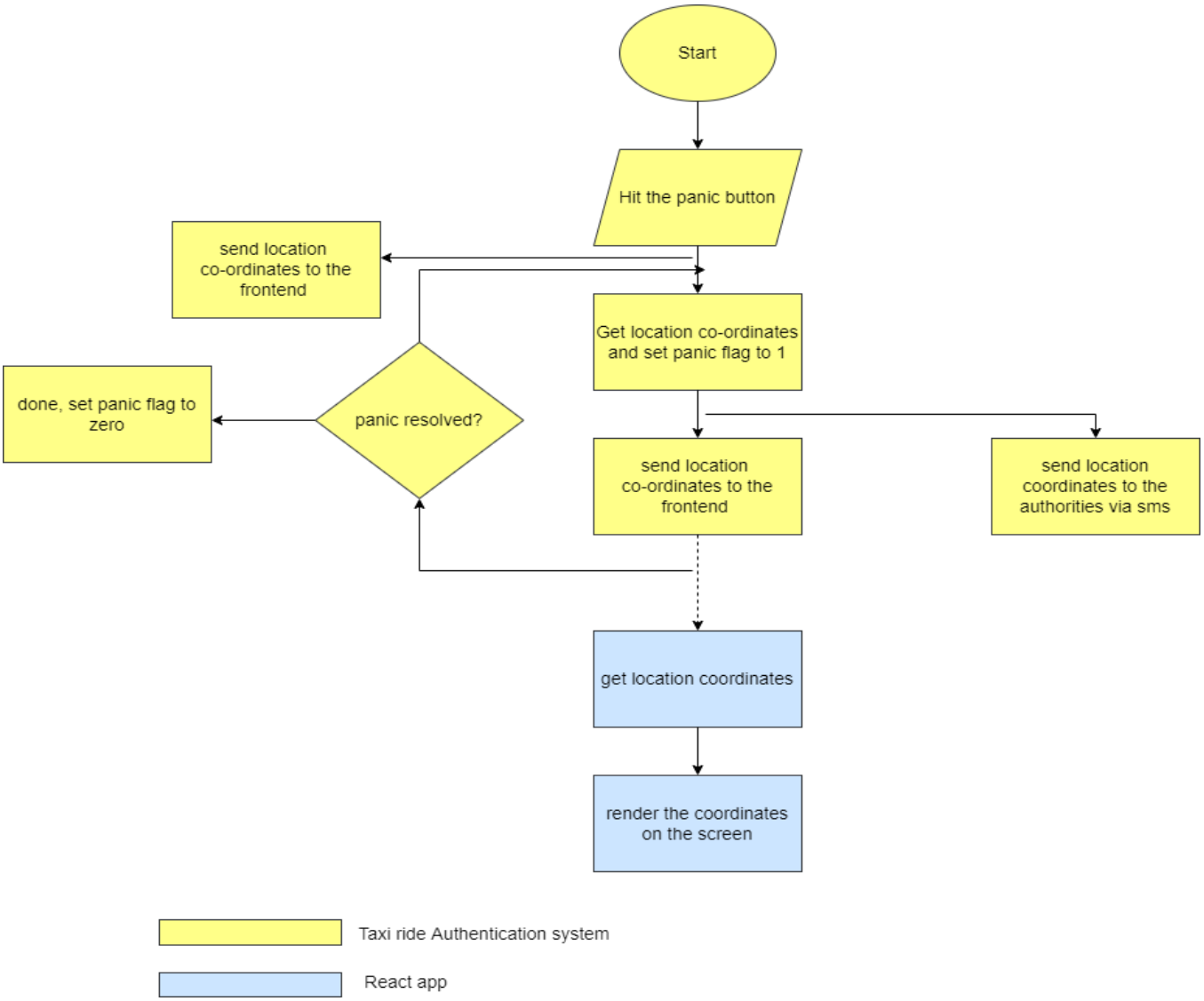


Figure 3.18 raising a panic

As shown above, when the passenger or driver hits the panic button, the program fetches the GPS co-ordinates via the universal asynchronous receiver transmitter (UART) number 2 where the gps sensor is connected. The values of the co-ordinates are read as a string showing the latitude and longitude values. The values are then converted into a json string, where the device id and the time are appended to the details and converted to json for sending via MQTT. At the same time the co-ordinates are sent via SMS to the responsible authorities (police) and a call is also made at the same time. When the frontend application receives the GPS co-ordinates, it decodes the co-ordinates and renders them on the screen for the admins to use. The program continues to send the co-ordinates to the frontend application every 5 seconds.

3.5.1.6.2 Resolving the Panic

Resolving a panic involved terminating the panic program and going to normal operation of the system. A Panic event was designed to be resolved by an administrator remotely from the frontend application after the situation has been attended to. The following flow chart shows how the design for resolving the panic was carried out.

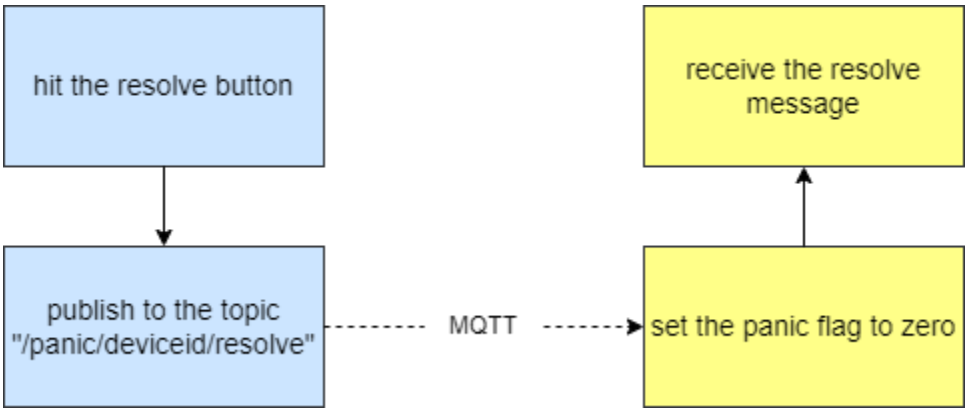


Figure 3.19-resolving a panic event

From figure 3.19, resolving a panic situation was designed to be carried out from the frontend application (shown in blue). When the resolve button is pushed, a MQTT topic is published to the broker, the broker receives the topic and forwards it to the device with the particular Id. Upon receiving the message, the device sets the panic flag to zero, meaning the panic loop is broken and the device can return to the normal operation.

3.5.1.7 Handling a robbery situation

This section describes the design of the program that the administrator uses to get information about a robbery that would have taken place in the taxi. The first step is to get the victim's name, and national id. These are used to search the registrar database for the victim's fingerprint details. Once the victim's fingerprints have been found, a search algorithm is used to find a matching fingerprint in the activity database. The records from the activity database include details that describe the time the victim was in the taxi, the taxi id, driver id and the time they went separate ways with the driver. The design is as shown in figure 3.20.

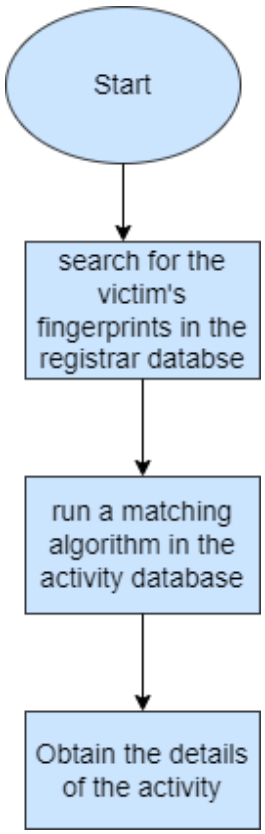


Figure 3.20-handling a crime

3.5.1.7.1 Design of the matching algorithm

The algorithm that was used for matching the fingerprints was developed using python with the help of scale-invariant feature transformation (SIFT) algorithm for identification of key points and flannBasedMatcher for fast nearest neighbor search.

The code for the matching algorithm is shown in appendix I. The program begins with reading the bmp image of the fingerprint to of the victim as shown in appendix I line 5. Appendix I-line 10 initiates a for loop that runs through the whole database of images performing the matching. Line 12 identifies key points in the fingerprint images (ridge and valley patterns) and stores the object in the variable sift. Line 13 and 14 compute the descriptors for the victim's image and the current image in the database. In line 16 the key points are matched and an object with the score results generated. The algorithm then calculates the score value using the formulae

$$\frac{length\ of\ match\ point}{keypoint} \times 100$$

The image with the largest score is then used to search the database for the activity record.

3.6 Conclusion

The design of the whole system involved mainly coding and circuit connections, the system required much attention in dealing with the fingerprint data and designing the algorithms for data conversion when transmitting fingerprint data over the internet. The implementation of the design methods and the results obtained are described in the following chapter.

4 Results and Discussion

4.1 Introduction

This chapter discusses the implementation of the design, testing of the system and the results obtained. Implementation and testing were based on the tasks and objectives that the system should meet.

4.2 Admin registration

Testing Administration involved the front-end application, the node server and the database and the test was successful as shown by the results in figure 4.1.

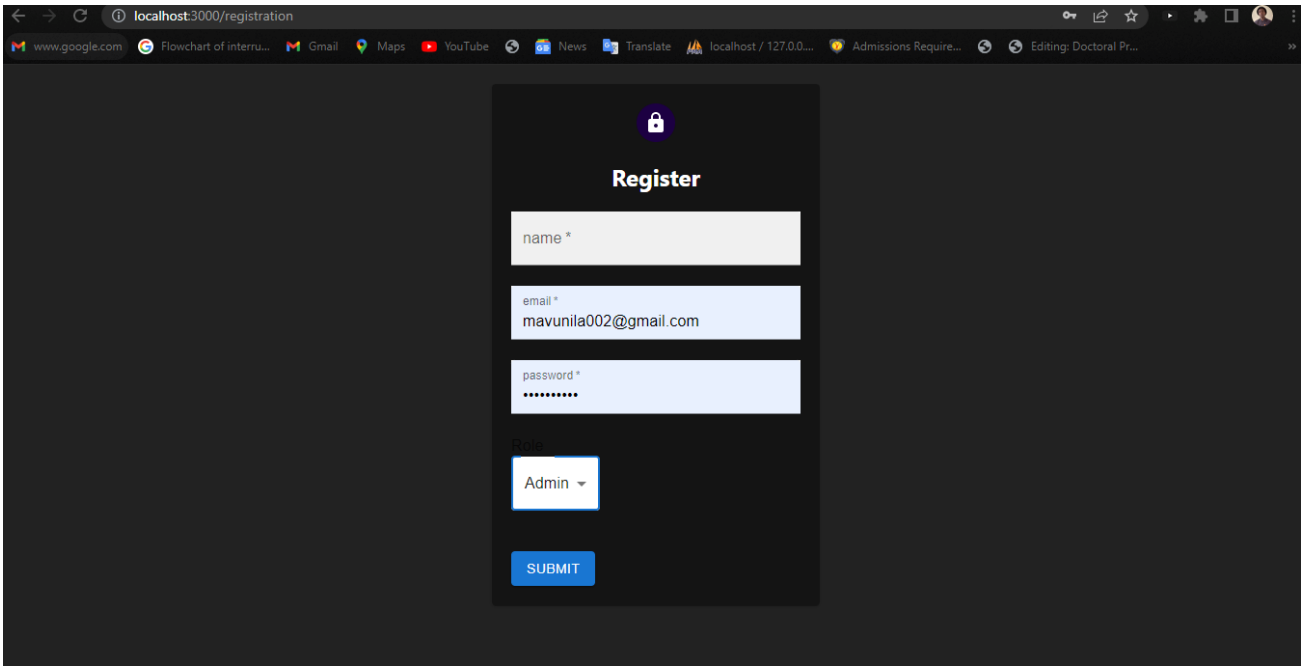


Figure 4.1 admin registration form being filled

In figure 4.1 is the admin registration form that was filled and submitted during the testing and the results shown below were obtained.

Figure 4.2 admin registration details in the database

Shown in figure 4.2 is the system database with the records of the administrator after registration. The server responded with status 200 ok after testing and the response was displayed on the front-end application as shown in figure 4.3.

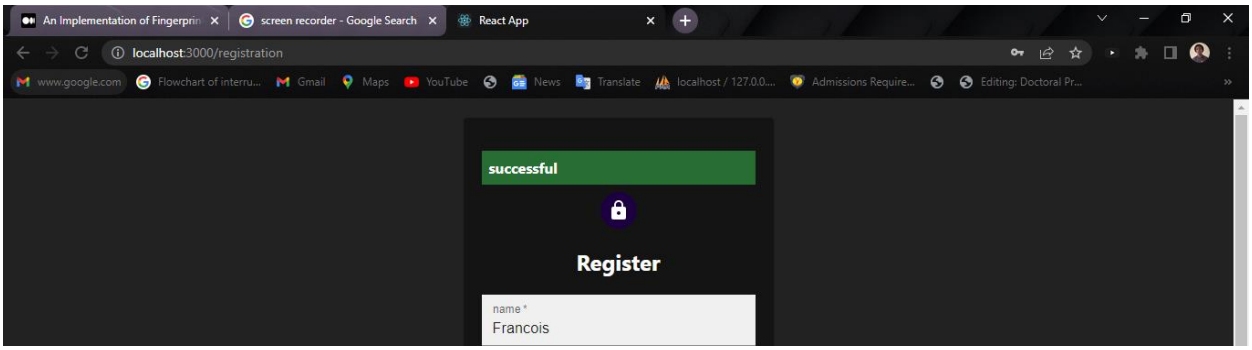


Figure 4.3message on successful registration

4.3 Admin login

The registered administrator was allowed to login first with incorrect details and an error message was raised and then when correct details were used the admin received a success message as shown in figure 4.4.

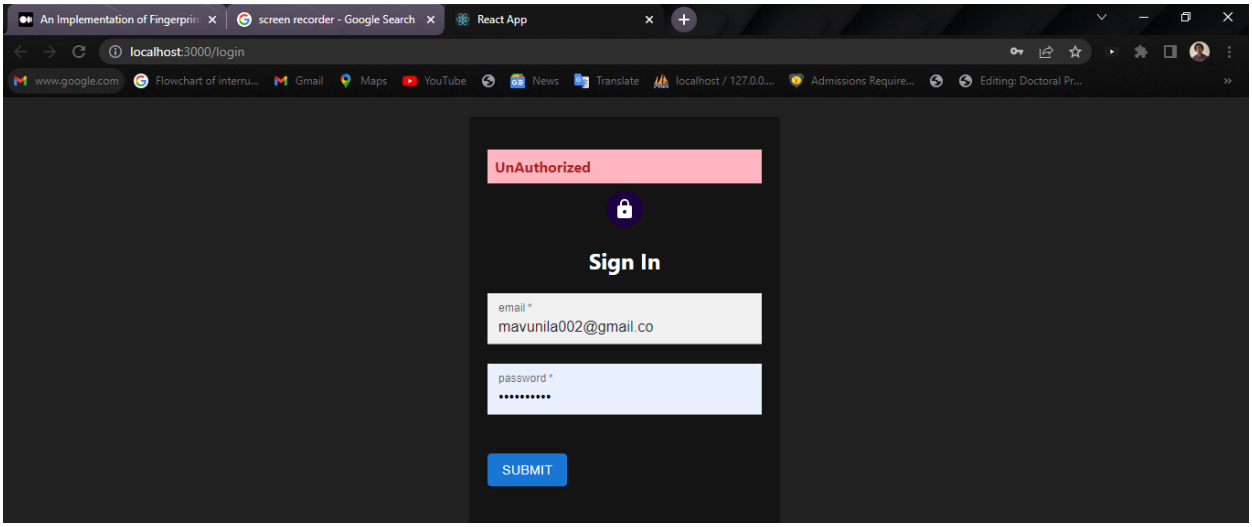


Figure 4.4 login with wrong password

4.4 Driver registration

To test driver registration, the driver details were filled in the form and sent over to the node.js server and the following results were obtained.

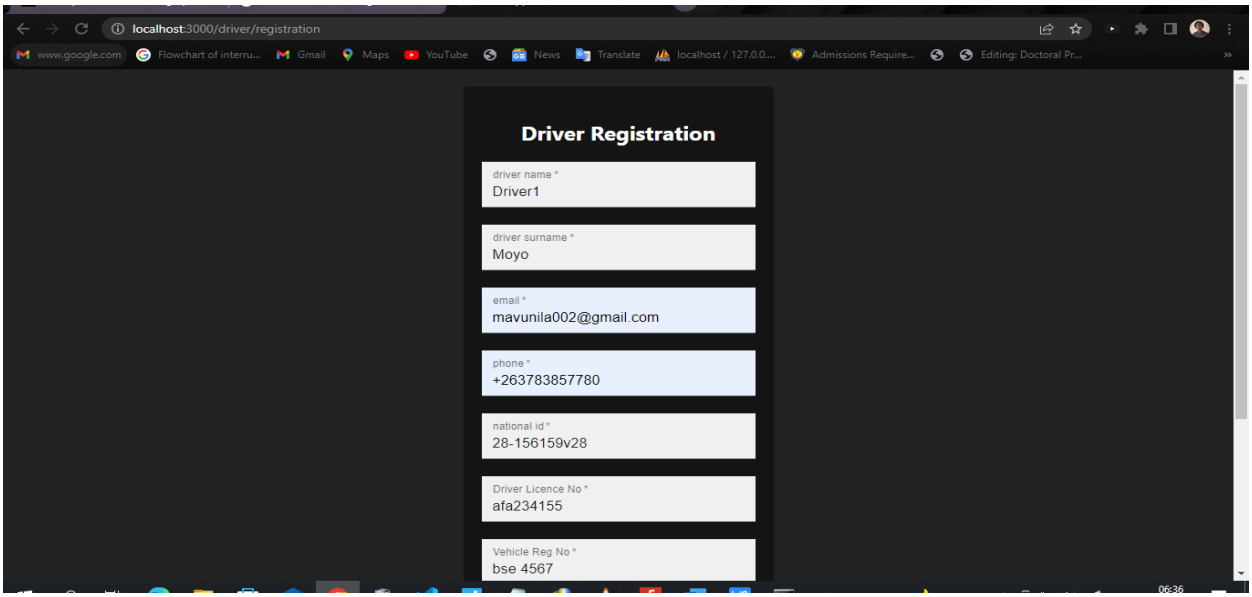


Figure 4.5 driver registration form

Figure 4.6 shows the server's response upon submitting the details and the driver's details after adding the fingerprint of the driver.

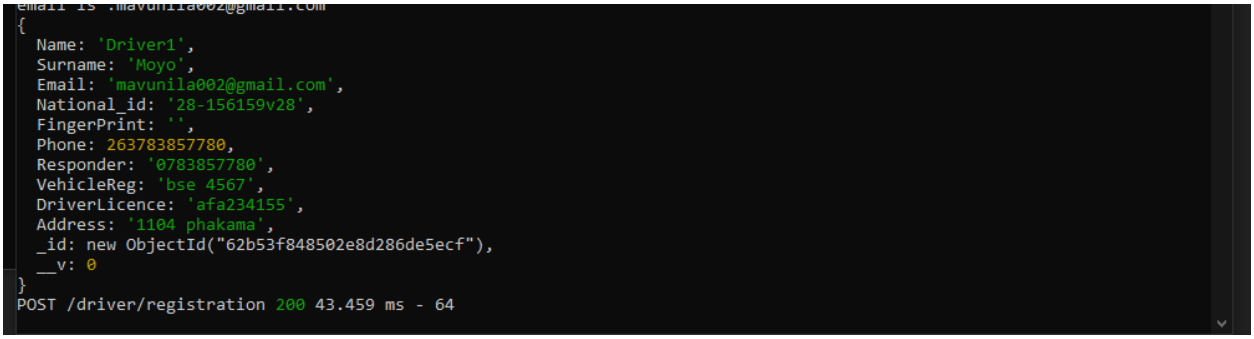


Figure 4.6 driver registration server side, with 200 ok status


```
_id: ObjectId('62a0776143621323abcfa45d')
Name: "Francois"
Surname: "Mavunila"
Email: "mavunila002@gmail.com"
National_id: "28-156159v28"
FingerPrint: "WzMsIDMsIDk1LCAxNywgMCwgMSwgMzIsIDEsIDEzOSwgMCwgMCwgMCwgMCwgMCwgMCwgMC…"
Phone: 263783857780
Responder: "078928928"
VehicleReg: "86276287262"
DriverLicence: "2627265267"
Address: "1104 phakama"
__v: 0
```


Figure 4.7 driver's details in the database

4.5 Capturing details at the beginning of the trip

Testing this objective involved capturing passenger's fingerprint and sending to the server, checking also the number of passengers in the taxi to see if the taxi is full. The results are shown in figure 4.8 explained below.

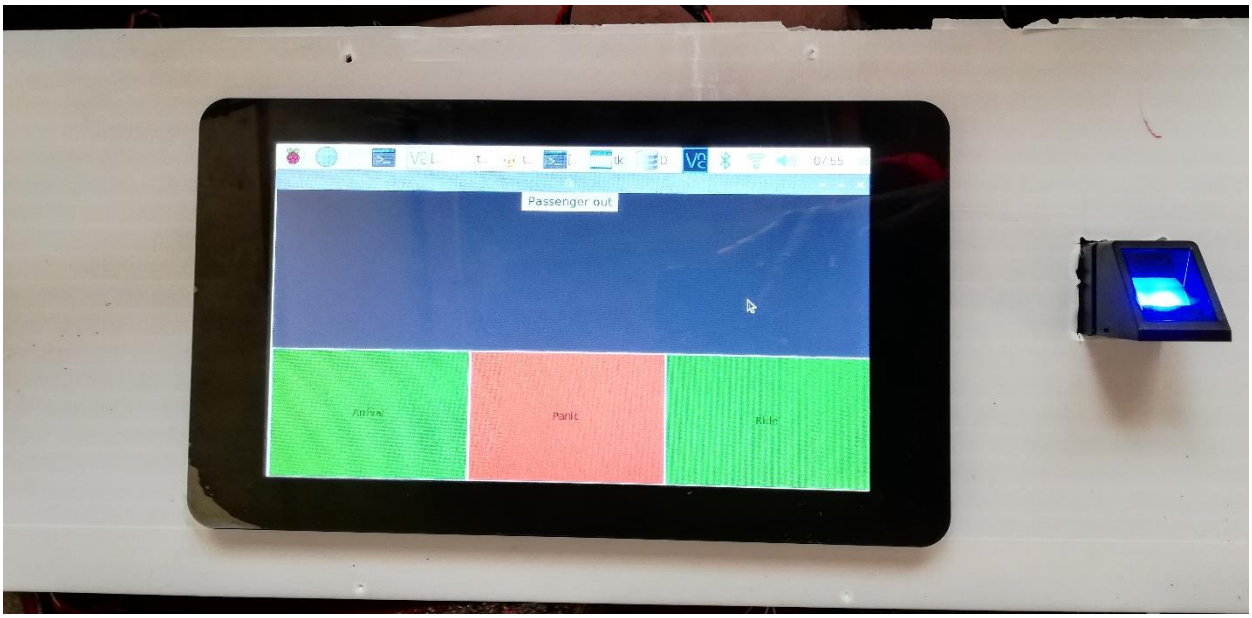


Figure 4.8 driver's dashboard with the HMI and the driver's fingerprint

The driver's dashboard with the HMI and the driver's fingerprint sensor. The GUI was tested successfully and used the three buttons to interact with the system. During testing, the ride button was pushed and the details of the passenger captured and successfully sent over MQTT to the server. The pictures below show the results obtained during the riding the taxi.


```
LCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAw
LCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAzLCAzLCA3NiwgMzAsIDAsIDEsIDMyLCAxLCAxNDgsIDAsIDAs
IDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAsIDAs
IDAsIDAsIDAsIDAsIDAsIDAsIDAsIDgsIDAsIDYsIDAsIDExNSwgMCwgNDgsIDMsIDAsIDAsIDYzLCAyNDAsIDIwNywgNjMsIDI1NSwgMjA3LCAyNTUsIDE5
MSwgMjM4LCAxNzAsIDE3MCwgMTcwLCAxNzAsIDE3MCwgMTcwLCAxMDEsIDg1LCA4NSwgODUsIDg1LCA2OSwgNjgsIDY5LCA0LCA2NCwgMCwgMCwgMCwgMCwg
MCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwg
MCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgMCwgNDAsIDI1LCA4NiwgNjIsIDc5LCAxNTcsIDY4LCAxMjYsIDkyLCA0NCwgMzAsIDIyMiwgMjAsIDUw
LCAxNTksIDEyNiwgOTEsIDE4MCwgMzIsIDE1OCwgMjcsIDE4MSwgMzMsIDk0LCAxOSwgNTYsIDIyMiwgNjIsIDM4LCAxODYsIDIyNCwgMTU4LCAyOSwgNjIs
IDk0LCAzMCwgNTcsIDY2LCAyNCwgMTI2LCAyMiwgNjYsIDkyLCAxMjYsIDg3LCAyMiwgMTk1LCAxMjcsIDUxLCAxNTYsIDIxNCwgOTUsIDI3LCA0NiwgMzYs
IDI1NSwgODEsIDE5MCwgMjI3LCAxOTEsIDEwMiwgNjMsIDEwMCwgMTI3LCA3OSwgNDksIDIyMSwgMTg4LCA3MiwgNTEsIDI3LCA5MiwgOTYsIDE2MCwgMTM1
LCAxMjIsIDk5LCAxNjMsIDI5LCAyNTAsIDE2LCAxNjMsIDk3LCA1OCwgMjMsIDE2OSwgMTAwLCAxMjIsIDE4LCAxNzEsIDMxLCA5MCwgMTksIDMyLCAxOTYs
IDU5LCA2MSwgNTUsIDE0LCA4OSwgNTUsIDYxLCAzNywgMjQ5LCA2MCwgMTc5LCAxMzgsIDU1LCA2MCwgNTMsIDEyLCAxNTEsIDQ5LCA2MCwgMzQsIDIzLCA1
MSwgNjAsIDEwMiwgMjE1LCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAwLCAw
LCAwL
```

Figure 4.9 are the results obtained of the passenger's fingerprint template upon riding the taxi, the fingerprint was encoded in base64. The result of the database containing the fingerprint details, time of riding and the ride flag and the depart flag is shown in figure 4.10.


```
_id: ObjectId('62b54e85e57e6ff26a124731')
DeviceId: "1010"
PassFingerPrint: "WzMsIDMsIDg4LCAzMiwgMjgsIDEsIDU3LCAxLCAxMzQsIDAsIDAsIDAsIDAsIDAsIDAsID…"
DepartureTime: "06/24/2022, 07:09:50"
ArrivalTime: ""
Departed: true
Arrived: false
__v: 0
```


Figure 4.10 passenger details in the database upon riding l

Figure 4.10 shows the passenger details were saved successfully with the departure time and the departure flag set. The ArrivalTime and the Arrived flag were set to false indicating the passenger has not left the taxi.

4.6 Capturing details at the end of the trip

The passengers were successfully captured and sent to the server upon arrival as shown by figure 4.11, the arrival flag was updated to true upon arrival and the arrival time was set as well.


```
_id: ObjectId('62b54ec7062b34808cfc6ea2')
DeviceId: "1010"
PassFingerPrint: "WzMsIDMsIDcxLCAzMywgNzYsIDEsIDc4LCA
DepartureTime: "06/24/2022, 07:42:06"
ArrivalTime: "06/24/2022, 07:45:49"
Departed: true
Arrived: true
__v: 0
```


Figure 4.11 arrival details successfully sent to the database.

4.7 Testing the Panic Mode

When the panic button was pushed, the gps co-ordinates were sent to the Front-end application and rendered as shown in figure 4.12.

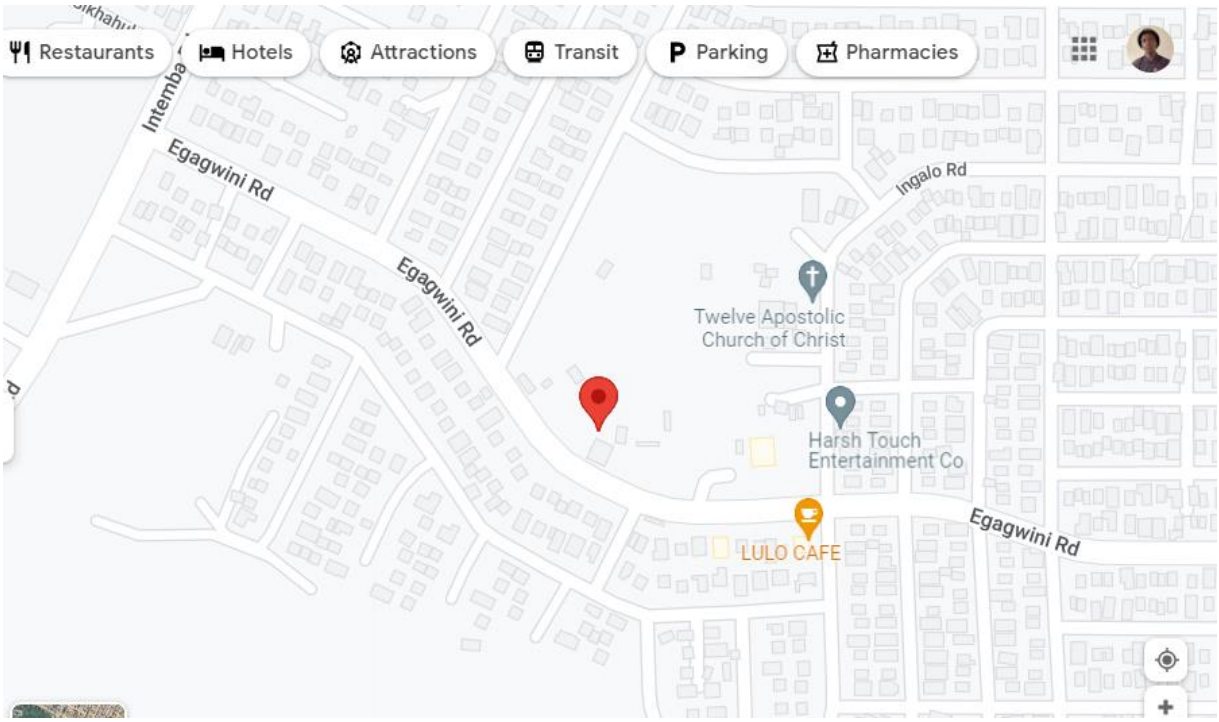


Figure 4.12 Panic mode

The System managed to send an SMS when the panic button was pushed, this is shown in figure 4.13 below.

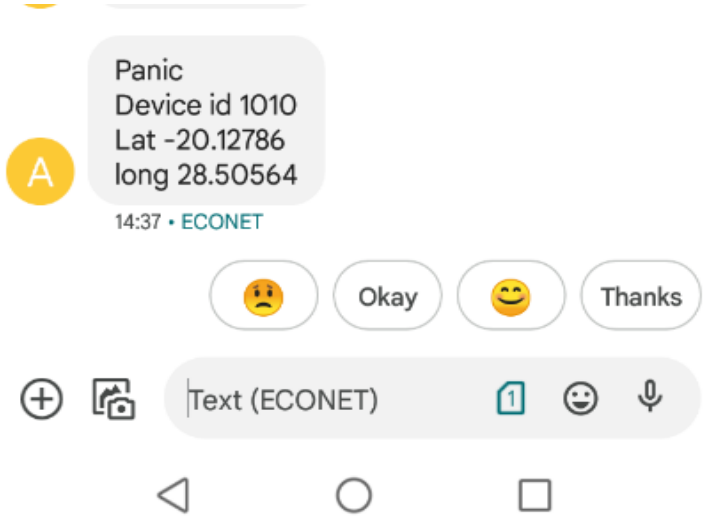


Figure 4.13 message sent on panic

5 Conclusions and Recommendations

5.1 Introduction

This chapter seeks to discuss the areas of improvement for the project, limitations of the project and bring about the conclusion of the project documentation.

5.2 Discussion

The system was designed and implemented successfully. The results obtained and presented in chapter 4 show that the system objectives were met to a greater extent. The system managed to achieve the following:

- The driver's details and the driver's fingerprints were successfully captured and sent to the database.
- The System managed to read the driver's and the passenger's fingerprint and send them to the server for processing.
- The system managed to authorize the driver on trip beginning and trip end.
- The system was designed successfully to check the number of passengers using seat sensors
- A web application was successfully developed
- The system managed to track the location of the taxi in panic mode and to send the SMS.

The system design and implementation proved that the system can be implemented in real life.

5.3 Limitations and Challenges

The greatest challenged faced during the system development was the development of the fingerprint matching algorithm. The fingerprint matching algorithms used by the police and the registrar are not known to the general population, hence creating an algorithm that does the same thing was a challenge.

The system was limited in that we could not get access to the actual registrar database and the registrar fingerprint searching algorithms. Another limitation was the use of an optical sensor that was robust and could not be used to check whether the person is alive or not.

5.4 Suggestions for further improvement

Instead of using an optical fingerprint sensor, the system can make use of the capacitive fingerprint sensor that is rather smaller and allows for checking whether the person is alive or not. The system

can also be integrated with the mobile app so as to take advantage of the already existing ride sharing safety features and add the fingerprint on top.

5.5 Conclusion

The project was designed, implemented and tested successfully. The objectives were met, though for the project to meet the market standard, some additional features should be added especially on securing the fingerprint data of the passengers.

6 References

[1] E. T. Luana B, "MAKING TAXIS SAFER," European Transport Safety Council, 2017.

[2] H. Ndhlovu, "Tuverl," Tuverl Inc, 2020. [Online]. Available: https://www.tuverl.com/blog/a-look-at-public-transportation-in-zimbabwe/. [Accessed 19 December 2021].

[3] C. CHICAGO, "CBS NEWS AND STATIONS," 2022.

[4] P. Nyathi, "Crackdown on pirate taxis, vendors intensifie," The Herald, Zimbabwe, 2022.

[5] A. Chaparadza, "How Safe Is It To Use Vaya Africa," TechZim, Bulawayo, 2018.

[6] M. J. Smith, "Robbery of Taxi Drivers," Office of Justice Programs, 2005.

[7] F. a. V. W. M. C. N. J. D. a. I. Hirschhorn, "Public Transport Governance',," *The Routledge Handbook for Public Transport*, no. 10.4324/9780367816698, p. 21–32, 2021.

[8] A. Iimi, R. M. Humphreys and Y. E. Mchomvu, "Modal Choice between Rail and Road Transportation : Evidence from Tanzania," *Policy Research Working Papers*, no. 8174, 2017.

[9] H. K. L. K. F. N. &. G. C. Shuyang Zhang, "Metro system disruption management and substitute bus service: a systematic review and future directions," *Transport Reviews*, no. 10.1080/01441647.2020.1834468, pp. 230-251, 2021.

[10] "Conserve Energy Future .," [Online]. Available: https://www.conserve-energy-future.com/benefits_of_public_transportation.php#:~:text=Public%20transportation%20is%20a%20form,dominate%20public%20transportation%20between%20cities.. [Accessed 27 May 2022].

[11] L. I. AndrésGómez, "Optimal prices for ridesourcing in the presence of taxi, public transport and car competition," *Transportation Research Part C: Emerging Technologies*, vol. 137, no. 103591, 2020.

[12] R. A. Mundy, "Why TNCs Will Be Regulated," Midwest transportation center , Iowa State University. Institute for Transportation ;, 2018.

[13] Chan, Nelson & Shaheen, Susan, Ridesharing in North America: Past, Present, and Future.Transport Reviews, vol. J, (2017).

[14] R. H. a. R. Metcalfe, "The Ridesharing Revolution: Economic Survey and Synthesis," Oxford University P, 2017.

[15] A. Sundararajan, "The Sharing Economy: The End of Employment and the Rise of Crowd-Based Capitalism," Cambridge: MIT Press, 2016.

[16] N. Y. a. S. Ravi, The Current and Future State of the, New Delhi : Brookings India IMPACT Series N, 2017.

[17] M. K. K. &. Đ. Slavuli, "The Evolution of Urban Transport - Uber," Sibenik, 2017.

[18] Turvel, "A look At Public Transportation In Zimbabwe," Tuverl Inc, Washington DC, USA, 2020.

[19] V. I. Tanyanyiwa, "Environment friendly and sustainable transportation in Harare," *Journal of Urban Planning, Landscape & environmental Design,* pp. 37-46., 2018.

[20] M. f. A. T. (. &. p. B. S. Pvt., "Market feasibility study of e-intracity buses in Harare," 2021.

[21] L. Mujuru, "Pirate Taxis Flourish in Zimbabwe's Capital As War Between Police and Drivers Escalates," Global Press Journal, 2017.

[22] N. Statt, "Uber's first ever safety report discloses 3,045 sexual assaults and nine murders in the US last year," The Verge, 2019.

[23] C. E. Kamais, "EMERGING SECURITY RISKS OF E-HAIL," *International Journal of Security, Privacy and Trust Management* , vol. 8, p. 3, 2019.

[24] K. E. Katie Mettler, "A student thought she had gotten into her Uber, police say. Hours later, hunters found her body," The washington post, WashingtonDC, 2018.

[25] P. G. S. T. W. Yanli Tang, "Gender-Related Operational Issues Arising from On-Demand Ride-Hailing Platforms: Safety Concerns and System Configuration," *Production and operations management,* vol. 30, no. 10, pp. 3481-3496, 2021.

[26] "REPORTED LIST OF INCIDENTS INVOLVING UBER AND LYFT," Atchison Arts, 26 July 2022. [Online]. Available: https://www.atchisontransport.com/blog/reported-list-of-incidents-involving-uber-and-lyft/. [Accessed 28 May 2022].

[27] M. Phiri, "Honda Fit has become the vehicle of choice for criminals in Zimbabwe," Mil and Guardian, Harare, 2022.

[28] N. Tshili, "plain robberies committed by Honda Fit drivers," Bulawayo, 2017.

[29] T. Muleya, "Gang of 7 kills a taxi driver," Beitbridge, 2017.

[30] D. C. Stevens, "Effectiveness of Taxi Partitions: Baltimore, Maryland, Case Study," *Transportation Research record.*

[31] J. R. S. C. Stevens, "Effectiveness of Taxi Partitions: Baltimore, Maryland, Case Study," *Transportation Research Record Journal of the Transportation Research Board*, no. 10.3141/1731-09, 2017.

[32] "Evaluation of Sample Taxicab Security Cameras," *Journal of Transportation Technologies* , vol. 7, no. 4, 2017.

[33] J. J. L. L. ,. R. S. ,. S. Haoran ZhangChen, "Mobile phone GPS data in urban ride-sharing: An assessment method for emission reduction potential," *Applied Energy* , p. 269, 2020.

[34] A. Benjamin, "CNET TEC," 2016.

[35] C. Net, "cnet.com," C Net, [Online]. Available: https://www.cnet.com/tech/mobile/how-ubers-new-safety-features-want-to-fight-murders-kidnappings-and-other-horrible-things/. [Accessed 10 11 2021].

[36] S. Shepard, "Lyft Adds New Security Measures for Riders," Campus Security & Life Safety, 2019.

[37] H. S. I. I. T. U. V. Africa?, "Alvine Chaparadza," 2018.

[38] L. O. R. H. L. Eric H. Holder, "The Fingerprint Sourcebook o," *Office of Justice Programs.*

[39] Y. Wate, "Different Types of Fingerprint Scanners," TECHPP, 2019.

[40] l. ada, "Adafruit Optical Fingerprint Sensor," Adafruit Industries, 2022.

[41] . Co-Operation With the Working Group RFID of the ETP EPOSS Internet of Things in 2020 Roadmap for the Future Version 1.1, May 2008..

[42] B. Mishra and A. Kertesz, "The Use of MQTT in M2M and IoT Systems," *A Survey*, vol. Volume 8, pp. 1-8, 04 November 2020.

[43] T. P. &. Z. M. Kumar S, "Internet of Things is a revolutionary approach for future technology enhancement," *Journal of Big Data*, vol. 6, no. 111, 2019.

[44] H. A. P. &. K. K., "Secure-MQTT: an efficient fuzzy logic-based approach to detect DoS attack in MQTT protocol for internet of things," *EURASIP Journal on Wireless Communications and Networking*, vol. 6, no. 90, 2019.

[45] D. M. H. A. a. M. Z. Musliyana Z, "Improvement of Data Exchange Security on HTTP," *Journal of Physics: Conference Series*, vol. 1019, no. 019 012073, pp. 25-27, 2017.

[46] P. C. K. H. Sudha Ellison Mathe A and V. S, "A Review on Raspberry Pi and its Robotic Applications," *International Conference on Artificial Intelligence and Signal Processing (AISP)*, vol. 2, no. 21725264, 2022.

[47] J. Howirtz, "Raspberry Pi 4 hits stores with faster CPU, 4K video, and RAM options," VentureBeat, 23 June 2019. [Online]. Available: https://venturebeat.com/2019/06/23/raspberry-pi-4-hits-stores-with-faster-cpu-4k-video-and-ram-options/. [Accessed 30 May 2022].

[48] M. Jena, "MongoDB Storage for Efficient Structuring of Data Simplified 101," 2022.

7 Appendix

Appendix A: Fingerprint code - initializing the fingerprint sensor

```
import tkinter as tk
from mymodules import mqttConnect
from mymodules import setMessage
from mymodules import ride
from mymodules import Driver
from mymodules import Passenger
from mymodules import doorlock
import threading

doorlock.doorlock("lock")
def test():
    print("working")
    setMessage.setMess("hgfd")
window = tk.Tk()
frame = tk.Frame(window)
frame.pack()

bottomframe = tk.Frame(window)
bottomframe.pack( side = tk.BOTTOM )
setMessage.setFrame(frame)
window.geometry('850x580')
window.config(bg='#222222')

Ride =  tk.Button(bottomframe, text="Ride",command=lambda:thread-
ing.Thread(target=ride.rideAuth).start(),activeback-
ground="blue",bg="green",height=10,width=30)
Arrival =  tk.Button(bottomframe, text="Arrival",command=lambda:thread
ing.Thread(target=ride.arrivalAuth).start(),activeback-
ground="blue",bg="green",height=10,width=30)
Panic = tk.Button(bottomframe, text="Panic",command="",activeback-
ground="blue",bg="red",height=10,width=30)
```

```
Ride.pack(side=tk.RIGHT)
Arrival.pack(side=tk.LEFT)
Panic.pack()
mqttConnect.connectMqtt()
window.mainloop()
```

Appendix B: GPS

```
import serial                #import serial pacakge
from time import sleep
import webbrowser            #import package for opening link in browse
import sys                   #import system package

gpgga_info = "$GPGGA,"
ser = serial.Serial ("/dev/ttyAMA2")              #Open port with baud
GPGGA_buffer = 0
NMEA_buff = 0
lat_in_degrees = 0
long_in_degrees = 0
def GPS_Info():
    global NMEA_buff
    global lat_in_degrees
    global long_in_degrees
    nmea_time = []
    nmea_latitude = []
    nmea_longitude = []

    nmea_time = NMEA_buff[0]                    #extract time from GPG

    nmea_latitude = NMEA_buff[1]                #extract latitude from

    nmea_longitude = NMEA_buff[3]               #extract longitude fro


    print("NMEA Time: ", nmea_time,'\n')
    print ("NMEA Latitude:", nmea_latitude,"NMEA Longitude:", nmea_lon

    lat = float(nmea_latitude)                  #convert string into f
calculation
    longi = float(nmea_longitude)               #convertr string into

    lat_in_degrees = convert_to_degrees(lat)    #get latitude in degre
```

```
35     degrees = int(decimal_value)
36      mm_mmmm = (decimal_value - int(decimal_value))/0.6
37      position = degrees + mm_mmmm
38      position = "%.4f" %(position)
39      return position
40
41
42
43
44
45 def getCoordinates():
46     global NMEA_buff
47     global lat_in_degrees
48     global long_in_degrees
49     try:
50         while True:
60             received_data = (str)(ser.readline())                    #r
61 received
62             GPGGA_data_available = received_data.find(gpgga_info)
63             if (GPGGA_data_available>0):
64                 GPGGA_buffer = received_data.split("$GPGGA,",1)[1]
65                 NMEA_buff = (GPGGA_buffer.split(','))
66
67                 GPS_Info()
68                 print("lat in degrees:", lat_in_degrees," long in degr
69 long_in_degrees, '\n')
70                 map_link = 'http://maps.google.com/?q=' + lat_in_degre
71 long_in_degrees    #create link to plot location on Google map
72                 break
73
74
75         coordinates={
76             "latitude": lat_in_degrees,
77             "longitude": long_in_degrees
78         }
79         return coordinates
80     except Exception as e:
81         print('Operation failed! in getcord')
82         print('Exception message: ' + str(e))
83         #exit(1)
84         coordinates={
85             "latitude ": 0,
86             "longitude": 0
87          }
88         return coordinates
```

Appendix C: Server.js

```
1   require('child_process').fork('mqtt.js');
2
3   const http = require('http');
4
5   const app = require('./app');
6
7   const port = process.env.PORT || 5003;// assign port
8
9   const server = http.createServer(app);
10
11  server.listen(port);
```

Appendix D: App.js

```
1   const express = require('express');
2   const morgan = require('morgan');
3   const bodyParser = require('body-parser');
4   const mongoose = require('mongoose');
5   const cookieParser = require('cookie-parser');
6   const app = express();
7   mongoose.connect('mongodb://0.0.0.0:27017/Taxanda_database')
8   .then(() => console.log("Database connected!"))
9   .catch(err => console.log(err));;
10
11  const userRoutes = require('./routes/user')
12
13  const driverRoutes = require('./routes/driver')
14
15  app.use(bodyParser.urlencoded({extended: false}));
16  app.use(bodyParser.json());
17  app.use('/user',userRoutes);
18  app.use('/driver',driverRoutes);
19  app.use((req,res,next)=>{
20      console.log('here')
21      const error = new Error('Not found');
22      error.status = 404;
23      next(error);
24  })
25  module.exports = app;
```

Appendix E: MQTT Node server client code

```
var mqtt = require('mqtt');
const driver = require('./models/driver_model.js');
const activity = require('./models/activity.js');
const mongoose = require('mongoose');
const { response } = require('express');
mongoose.connect('mongodb://0.0.0.0:27017/Taxanda_database')
.then(() => console.log("Database connected on child process!"))
.catch(err => console.log(err));;

var options = {
    host: '9c641868feef4f288cb222c3d38b905e.s1.eu.hivemq.cloud',
    port: 8883,
    protocol: 'mqtts',
    username: 'Francois',
    password: '12345Fra'
}


var client = mqtt.connect(options);

client.on('connect', function () {
    console.log('Connected');
});

client.on('error', function (error) {
    console.log(error);
});

client.on('message', function (topic, message) {
    //Called each time a message is received
    if(topic=='driv/register/sendFP'){
        console.log( message.toString());
        var messageObj = JSON.parse(message);
        console.log("device id"+messageObj.device_id)
        console.log("driver id"+messageObj.driver_id)
        console.log("data"+messageObj.data)
        driver.findOne({National_id:messageObj.driver_id})
        .exec()
        .then((_driver)=>{
            if(_driver){
               console.log("driver is"+_driver);
               _driver.FingerPrint =messageObj.data;

```

```
_driver.save()
                .then(result=>{
                    console.log("done, send flag to app now");
                    client.publish('registration', 'done');
                })
            }else{
                console.log("couldn't get particular driver");
                client.publish('registration', 'Wrong Driver Id');
            }

        })
        .catch(err=>{
            console.log("error is"+err);
            client.publish('registration', 'server error');
        })
    }else if(topic=="activity/1010/depart"){
        console.log( message.toString());
        var messageObj = JSON.parse(message);
        console.log("device id"+messageObj.device_id)
        console.log("data"+messageObj.data)
        console.log("rowId"+messageObj.rowId)
        const Activity = new activity({
            DeviceId : messageObj.device_id ,
            PassFingerPrint: messageObj.data,
            DepartureTime : messageObj.time,
            ArrivalTime :"",
            Departed : true,
            Arrived: false,
        });
        Activity.save()
        .then((response)=>{
          if(response){
              console.log("tell device success");
              var rowId=messageObj.rowId.toString();
              client.publish('activity/1010/depart/done',rowId );
          }else{
            console.log("ask device to try sending again");
          }
        })
        .catch(err=>{
            console.log(err);
        })
    }else if(topic=="activity/1010/arrival"){
        console.log( message.toString());
        var messageObj = JSON.parse(message);
        console.log("device id"+messageObj.device_id)
        console.log("data"+messageObj.data)
        activity.findOne({PassFingerPrint:messageObj.data})
        .exec()
        .then(_activity=>{
        if(_activity){
            console.log(_activity)
            console.log("device id"+messageObj.device_id)
```

```
97     console.log("data"+messageObj.data)
98           activity.findOne({PassFingerPrint:messageObj.data})
99           .exec()
100          .then(_activity=>{
101          if(_activity){
102              console.log(_activity)
103              console.log("device id"+messageObj.device_id)
104                 _activity.Arrived = true;
105                 _activity.ArrivalTime = messageObj.time;
106                 _activity.save()
107                 .then(response=>{
108                     if(response){
109                         console.log("tell device success");
110                         var rowId=messageObj.rowId.toString();
111                         client.publish('activity/1010/arrival/do
112 );
113                     }else{
114                         console.log("ask device to try sending a
115                     }
116                 })
117          }else{
118              console.log("failed to find one");
119          }
120
121          })
122          .catch(err=>{
123              console.log(err);
124          })
125      }else{
126          console.log("topic not defined")
127      }
128 });
129 client.subscribe('driv/register/sendFP');
130 client.subscribe('activity/1010/depart');
131 client.subscribe('activity/1010/arrival');
```

Appendix F: Creating database models

Activity schema

```
1   const mongoose = require('mongoose');
2
3   const ActivitySchema = mongoose.Schema({
4       DeviceId:{type: String, required: true},
5       PassFingerPrint: {type:String, required: true} ,
6       DepartureTime:{type: String,required: true, unique:true},
7       ArrivalTime: {type:String} ,
8       Departed : {type:Boolean},
9       Arrived : {type:Boolean},
10  });
11
12  module.exports = mongoose.model('Activity',ActivitySchema);
13
```

Driver Schema

```
1   const mongoose = require('mongoose');
2
3   const DriverSchema = mongoose.Schema({
4       Name :{type: String, required: true},
5       Surname: {type:String, required: true} ,
6       Email:{type: String,required: true, unique:true},
7       National_id: {type:String, required: true} ,
8       FingerPrint : {type:String},
9       Phone : {type:Number,required:true},
10      Responder: {type:String, required: true} ,
11      VehicleReg: {type:String, required: true} ,
12      DriverLicence: {type:String, required: true},
13      Address: {type:String, required: true}
14  });
15
16  module.exports = mongoose.model('Drivers',DriverSchema);
```

Administrator Schema

```
const mongoose = require('mongoose');

const UserSchema = mongoose.Schema({
    Name :{type: String, required: true},
    Email:{type: String,required: true, unique:true},
    Password: {type:String, required: true} ,
    refreshToken : {type:String},
    Role : {type:Number,required:true}
});

module.exports = mongoose.model('Users',UserSchema);
```

Appendix G: Web application design

Design of form

```
    <Grid >
         <Paper elavation={24} style={paperStyle}>
           <p ref={errRef} className={errMsg? "errmsg" : "offscreen"}
aria-live="assertive">{errMsg}</p>
             <form  noValidate >
                 <TextField id="filled-basic" variant="filled"
color="primary" label="email" placeholder='Enter email' value={email}
                   fullWidth required
                 onChange={e => setEmail(e.target.value)}
className={classes.root} inputRef={userRef} />
                 <br></br><br></br>
                 <TextField  id="filled-basic"
className={classes.root}  variant="filled" label="password"
                 placeholder='Enter password' type='password'
value={password} fullWidth required
                 onChange={e => setPassword(e.target.value)} />
             </form><br></br><br></br>
             <Button onClick={handleSubmit} variant="contained"
>Submit</Button>
         </Paper>
     </Grid>
```

Axios

```
const handleSubmit = (evt) => {
        evt.preventDefault();
        axios.post('/user/login', {
            email: email,
            password: password
          },
          {
            headers:{'Content-Type':'application/json'},
            withCredentials: true
          }
          )
          .then(function (response) {
            console.log(response);
            if(response.data){
              const accessToken = response?.data?.accessToken;
              const roles = response?.data?.role;
              setAuth({email, password, roles, accessToken});
              setEmail('');
              setPassword('');
              navigate(from,{replace: true});
            }else{
              alert('please check your username and password')
            }
          })
          .catch(function (err) {
            console.log(err);
            if(!err?.response){
              setErrMsg('No Server Response');
            }else if(err.response?.status==400){
              setErrMsg('Missing Username or Password');
            }else if(err.response?.status==401){
              setErrMsg('UnAuthorized');
            }else{
              setErrMsg('Failed Login')
            }
            errRef.current.focus();
          });
    }
useEffect(()=>{
  userRef.current.focus();
},[])
```

Appendix H: Driver Registration code

```
def enrollDriverFingerPrint(driver_id):
        print('Waiting for finger...')
        setMessage.setMess("Driver place your finger on the sensor")
        while ( d.readImage() == False ):
            pass
        d.convertImage(0x01)
        result = d.searchTemplate()
        positionNumber = result[0]
        if ( positionNumber >= 0 ):
            print('Template already exists at position #' + str(positi
            setMessage.setMess("your fingerprint is already enrolled,
delete the enroll fingerprint")
            return 301
        print('Remove finger...')
        setMessage.setMess("Remove Finger")
        print('Waiting for same finger again...')
        setMessage.setMess("place the same finger again")
        while ( d.readImage() == False ):
            pass
        print('Downloading image (this take a while)...')
        setMessage.setMess("we are acquiring the information, wait....
        imageDestination =  tempfile.gettempdir() + '/driver.bmp'
        d.downloadImage(imageDestination)
        print('The image was saved to "' + imageDestination + '".')
        d.createTemplate()
          positionNumber = d.storeTemplate()
        print('Finger enrolled successfully!')
        print('New template position #' + str(positionNumber))
        d.loadTemplate(positionNumber, 0x01)
        characterics = str(d.downloadCharacteristics(0x01)).encode()
        cursor.execute('INSERT INTO driver (driverId, deviceId,
template,image,date) VALUES(?, ?, ?, ?, ?)', (driver_id,
1010,template,imageDestination,currentDT ))
        dbconnect.commit();
        dat={
            "device_id":1010,
            "driver_id":driver_id,
            "data":dataBase64.decode()
        }
        setMessage.setMess("Done")
        print(dat)
        print(dataBase64)
        my_json_object = json.dumps(dat)
        mqttConnect.driverMqtt("driv/register/sendFP",my_json_object)
        return positionNumber
```

Appendix I: Fingerprint matching algorithm

```
import cv2
import numpy as np
import os

fingerprint_test = cv2.imread("./fingerprint/point_1.tiff")
cv2.imshow("Original", cv2.resize(fingerprint_test, None, fx=1, fy=1))
cv2.waitKey(0)
cv2.destroyAllWindows()

for file in [file for file in os.listdir("database")]:
    fingerprint_database_image = cv2.imread("./database/"+file)
    sift = cv2.SIFT_create()
    keypoints_1, descriptors_1 = sift.detectAndCompute(fingerprint_tes
    keypoints_2, descriptors_2 = sift.detectAndCompute(fingerprint_dat

    matches = cv2.FlannBasedMatcher(dict(algorithm=1, trees=10),
             dict()).knnMatch(descriptors_1, descriptors_2, k=2)
    match_points = []

    for p, q in matches:
        if p.distance < 0.1*q.distance:
         match_points.append(p)

    keypoints = 0
    if len(keypoints_1) <= len(keypoints_2):
      keypoints = len(keypoints_1)
    else:
      keypoints = len(keypoints_2)

    if (len(match_points) / keypoints)>0:
      print("% match: ", len(match_points) / keypoints * 100)
      print("Figerprint ID: " + str(file))
      result = cv2.drawMatches(fingerprint_test, keypoints_1, fingerpr
                               keypoints_2, match_points, None)
      result = cv2.resize(result, None, fx=1, fy=1)
      cv2.imshow("result", result)
      cv2.waitKey(0)
      cv2.destroyAllWindows()
      break;
```